

Statistical learning for spatio-temporal point processes: inference and testing

Jorge Mateu
University Jaume I, Castellón, Spain

Based on joint works with Abdollah Jalilian, Yao Xie, Zheng Dong and George Mohler

14th European Congress for Stereology and Image Analysis (ECSIA)

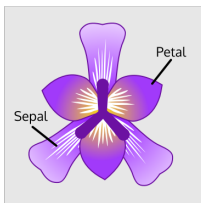
September 16, 2025

Presentation based on the following papers

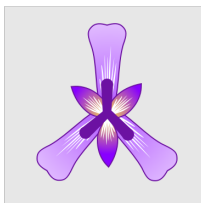
- Jalilian, A. and Mateu, J. (2023). Assessing similarities between spatial point patterns with a Siamese Neural Network discriminant model. **Advances in Data Analysis and Classification**, **17**, 21-42.
- Dong, Z., Zhu, S., Xie, Y., Mateu, J. and Rodriguez-Cortes, F. (2023). Non-stationary spatio-temporal point process modeling for high-resolution COVID-19 data. **Journal of the Royal Statistical Society C**, **72**, 368–386.
- Mohler, G. and Mateu, J. (2024). Second order preserving point process permutations. **Stat.** doi: 10.1002/sta4.558.
- Dong, Z., Mateu, J. and Xie, Y. (2025). Spatio-temporal-network point processes for modeling crime incidents with landmarks. **Submitted.**
- Platero, J., Walchessen, J., Kuusela, M. and Mateu, J. (2025). Neural likelihood inference for complex spatial points processes. **Submitted.**

1. Similarities between spatial point patterns with a Siamese Neural Network discriminant model

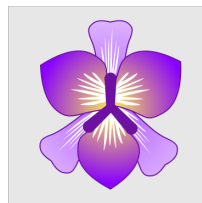
Fisher's or Anderson's iris dataset



Iris Versicolor



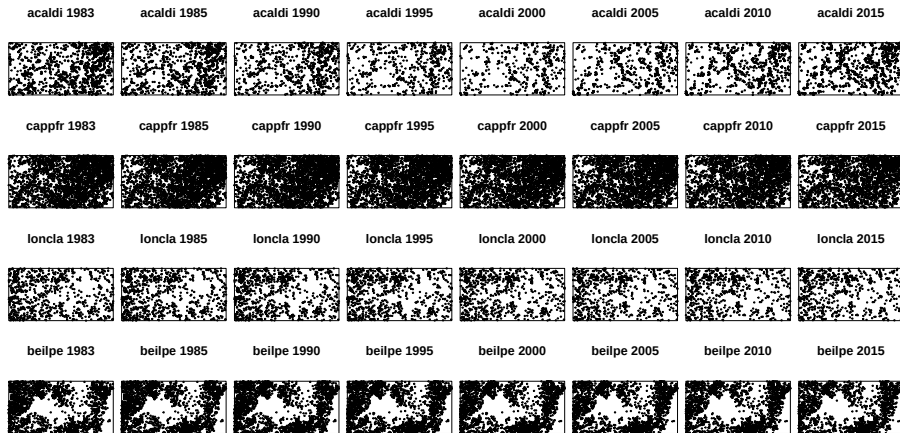
Iris Setosa



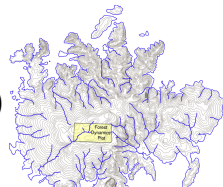
Iris Virginica

i	Sepal Length	Sepal Width	Petal Length	Petal Width	Species
1	5.1	3.5	1.4	0.2	setosa
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
50	5.0	3.3	1.4	0.2	
51	7.0	3.2	4.7	1.4	versicolor
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
100	5.7	2.8	4.1	1.3	
101	6.3	3.3	6.0	2.5	virginica
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
150	5.9	3.0	5.1	1.8	

BCI dataset



- $m = 130$ most abundant species
- $T = 8$ temporally dependent repetitions (censuses)
- $N = 1040$ spatial point patterns (1808725 alive trees)
- a discriminant model for spatial point patterns?



Dissimilarity function

- a dissimilarity (distance) function

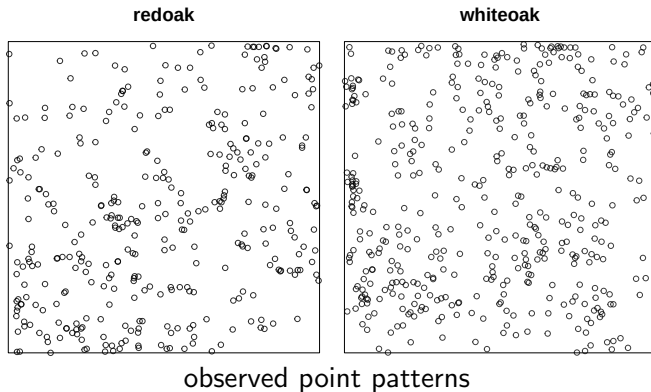
$$D : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$$

- good dissimilarity function

$$D(\mathbf{x}^{(s_i, t_i)}, \mathbf{x}^{(s_{i'}, t_{i'})}) = \begin{cases} \text{as small as possible} & s_i = s_{i'} \\ \text{as large as possible} & s_i \neq s_{i'} \end{cases}$$

- ▶ compares **distinctive features** between groups
 - ▶ allows for **variations** within groups
- comparing patterns versus comparing processes
 - ▶ pattern matching: similarities of observed point patterns
 - ▶ process matching: similarities of the generative point processes

Dissimilarity based on pattern matching



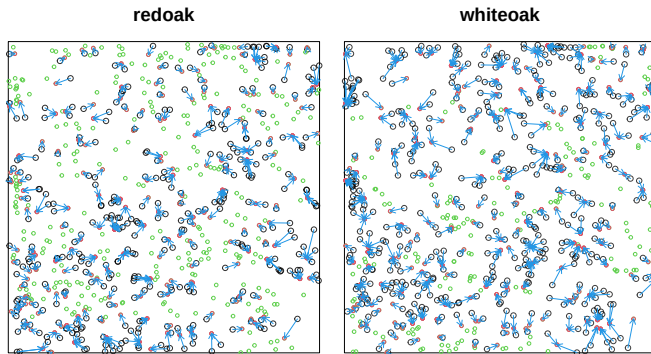
Dissimilarity based on pattern matching

redoak					whiteoak				
4	9	7	11	19	23	16	21	31	15
10	8	8	20	9	12	14	17	13	17
13	20	17	16	11	25	9	15	17	9
16	19	18	7	20	33	18	25	21	9
27	22	21	9	5	22	20	14	16	16

Alba-Fernández et al. (2016): dissimilarity of quadrat counts

$$D(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^m \left\{ \left(1 - \frac{n(\mathbf{x} \cap B_j)[n(\mathbf{x}) + n(\mathbf{x}')]}{n(\mathbf{x})[n(\mathbf{x} \cap B_j) + n(\mathbf{x}' \cap B_j)]} \right)^2 + \left(1 - \frac{n(\mathbf{x}' \cap B_j)[n(\mathbf{x}) + n(\mathbf{x}')]}{n(\mathbf{x}') [n(\mathbf{x} \cap B_j) + n(\mathbf{x}' \cap B_j)]} \right)^2 \right\}$$

Dissimilarity based on pattern matching



Cholaquidis, et al. (2017): dissimilarity of nearest neighbour distances

$$D(\mathbf{x}, \mathbf{x}') = \max \left\{ \sup_{\mathbf{u} \in \mathbf{x}} \inf_{\mathbf{v} \in \mathbf{x}'} \frac{\|\mathbf{u} - \mathbf{v}\|}{\text{diam}(W)}, \sup_{\mathbf{u} \in \mathbf{x}'} \inf_{\mathbf{v} \in \mathbf{x}} \frac{\|\mathbf{u} - \mathbf{v}\|}{\text{diam}(W)} \right\} \\ + |n(\mathbf{x}) - n(\mathbf{x}')|$$

Dissimilarity based on process matching

- $\mathbf{x}$ and $\mathbf{x}'$ are realizations of point processes X and X'
 - ▶ $\mathbb{P}_X = \mathbb{P}_{X'}$: $\mathbf{x}$ and $\mathbf{x}'$ are statistically indistinguishable; their differences are due to pure chance
 - ▶ $\mathbb{P}_X \neq \mathbb{P}_{X'}$: $\mathbf{x}$ and $\mathbf{x}'$ are expected to reflect some structural differences; their differences are results of some distinctive **features** in $\mathbb{P}_X$ and $\mathbb{P}_{X'}$
- a suitable summary statistic to quantify/reflect some **relevant** features of the underlying point processes

$$G : \mathcal{X} \rightarrow \mathcal{F}$$

- dissimilarity function based on distance of patterns in the feature space $\mathcal{F}$

$$D(\mathbf{x}, \mathbf{x}') = D_{\mathcal{F}}(G(\mathbf{x}), G(\mathbf{x}')) = \|G(\mathbf{x}) - G(\mathbf{x}')\|_{\mathcal{F}}$$

Examples of summary statistics

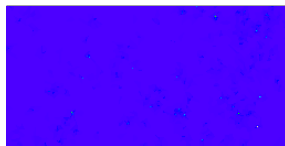
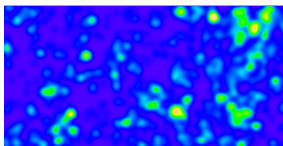
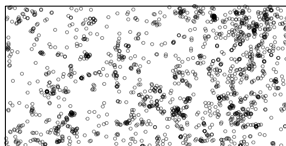
- general form of common summary statistics

$$G(\mathbf{x}) = \sum_{\substack{\neq \\ \mathbf{u}_1, \dots, \mathbf{u}_\ell \in \mathbf{x}}} h(\mathbf{u}_1, \dots, \mathbf{u}_\ell; \mathbf{x} \setminus \{\mathbf{u}_1, \dots, \mathbf{u}_\ell\})$$

- first order summary statistics

$$G(\mathbf{x}) = \hat{\rho}(\mathbf{u}; \mathbf{x}) = \sum_{\mathbf{v} \in \mathbf{x}} \frac{k(\mathbf{u} - \mathbf{v})}{\int_W k(\mathbf{u} - \mathbf{v}') d\mathbf{v}'}$$

feature space $\mathcal{F}$ consists of non-negative pixel images on W



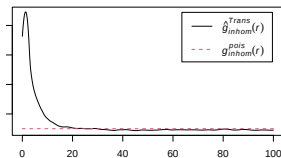
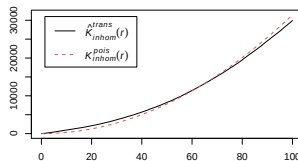
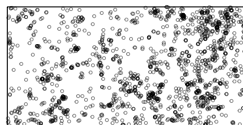
Examples of summary statistics

- second order summary statistics

$$G(\mathbf{x}) = \hat{K}(r; \mathbf{x}) = \sum_{\mathbf{u}, \mathbf{v} \in \mathbf{x}}^{\neq} \frac{\mathbb{1}[\|\mathbf{u} - \mathbf{v}\| \leq r]}{\hat{\rho}(\mathbf{u}; \mathbf{x}) \hat{\rho}(\mathbf{v}; \mathbf{x}) |W \cap W_{\mathbf{u}-\mathbf{v}}|}$$

$$G(\mathbf{x}) = \hat{g}(r; \mathbf{x}) = \frac{1}{2\pi} \sum_{\mathbf{u}, \mathbf{v} \in \mathbf{x}}^{\neq} \frac{k(r - \|\mathbf{u} - \mathbf{v}\|)}{\|\mathbf{u} - \mathbf{v}\| \hat{\rho}(\mathbf{u}; \mathbf{x}) \hat{\rho}(\mathbf{v}; \mathbf{x}) |W \cap W_{\mathbf{u}-\mathbf{v}}|}$$

feature space $\mathcal{F}$ consists of matrices with at least two columns



Examples of summary statistics

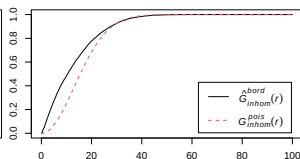
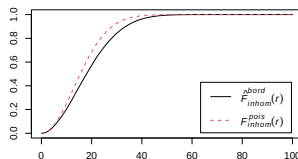
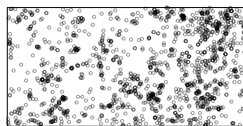
- inter-point distance summary statistics

$$G(\mathbf{x}) = \hat{F}(r; \mathbf{x}) = 1 - \frac{1}{\#(L \cap W_{\Theta r})} \sum_{\mathbf{v} \in L \cap W_{\Theta r}} \prod_{\substack{\mathbf{u} \in \mathbf{x} \\ \|\mathbf{u} - \mathbf{v}\| \leq r}} \left[1 - \frac{\inf_{\mathbf{u}'} \hat{\rho}(\mathbf{u}'; \mathbf{x})}{\hat{\rho}(\mathbf{u}; \mathbf{x})} \right]$$

$$G(\mathbf{x}) = \hat{G}(r; \mathbf{x}) = 1 - \frac{1}{\#(\mathbf{x} \cap W_{\Theta r})} \sum_{\mathbf{v} \in \mathbf{x} \cap W_{\Theta r}} \prod_{\substack{\mathbf{u} \in \mathbf{x} \setminus \{\mathbf{v}\} \\ \|\mathbf{u} - \mathbf{v}\| \leq r}} \left[1 - \frac{\inf_{\mathbf{u}'} \hat{\rho}(\mathbf{u}'; \mathbf{x})}{\hat{\rho}(\mathbf{u}; \mathbf{x})} \right]$$

$$G(\mathbf{x}) = \hat{J}(r; \mathbf{x}) = \frac{1 - \hat{G}(r; \mathbf{x})}{1 - \hat{F}(r; \mathbf{x})}$$

feature space $\mathcal{F}$ consists of matrices with at least two columns



Which summary statistic?

- which summary statistic should be used?
 - ▶ visual inspection
 - ▶ prior information

$$\hat{\rho}(\cdot; \mathbf{x}) \quad \hat{K}(\cdot; \mathbf{x}) \quad \hat{g}(\cdot; \mathbf{x}) \quad \hat{F}(\cdot; \mathbf{x}) \quad \hat{G}(\cdot; \mathbf{x}) \quad \hat{J}(\cdot; \mathbf{x})$$

- which norm/distance in the feature space?

$$D(\mathbf{x}, \mathbf{x}') = \|\hat{\rho}(\cdot; \mathbf{x}) - \hat{\rho}(\cdot; \mathbf{x}')\|_2 = \int_W [\hat{\rho}(\mathbf{u}; \mathbf{x}) - \hat{\rho}(\mathbf{u}; \mathbf{x}')]^2 d\mathbf{u}$$

$$D(\mathbf{x}, \mathbf{x}') = \|\hat{K}(\cdot; \mathbf{x}) - \hat{K}(\cdot; \mathbf{x}')\|_\infty = \sup_{0 \leq r \leq r_{\max}} \left| \hat{K}(r; \mathbf{x}) - \hat{K}(r; \mathbf{x}') \right|$$

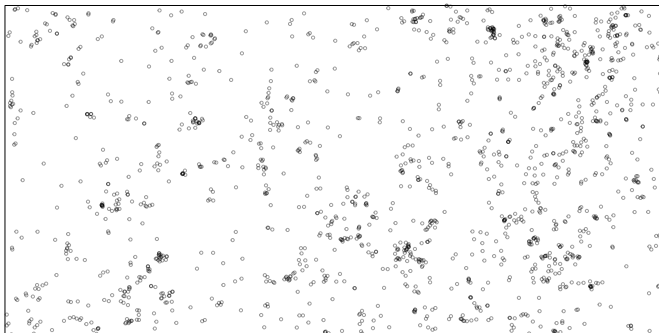
Mateu *et al.* (2015). On measures of dissimilarity between point patterns: classification based on prototypes and multidimensional scaling.

Biometrical Journal.

Feature extraction by CNN

- partition $W \subset \mathbb{R}^2$ into a $d_1 \times d_2$ regular grid of cells

$$W = \bigcup_{i=1}^{d_1} \bigcup_{j=1}^{d_2} B_{ij}$$



observed point pattern: $\mathbf{x}_W = \{\mathbf{u}_1, \dots, \mathbf{u}_{n(\mathbf{x})}\}$

observation window: $W = [0, 1000] \times [0, 500]$

Feature extraction by CNN

- partition $W \subset \mathbb{R}^2$ into a $d_1 \times d_2$ regular grid of cells

$$W = \bigcup_{i=1}^{d_1} \bigcup_{j=1}^{d_2} B_{ij}$$

3	1	0	3	1	0	2	0	0	0	1	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	1	3	6	0	3	3	1	2	1	0	0	1	1	5	1	0	0	4	7	2	0			
0	0	1	3	1	4	0	2	0	0	1	0	0	0	0	0	1	1	0	2	0	0	0	0	2	1	1	0	3	8	1	4	3	1	8	1	0	2	3	7	5	0	2	4	9	1	3	0			
2	1	7	2	0	0	2	0	0	2	0	0	1	9	0	0	0	0	0	1	0	3	0	1	3	1	0	0	0	0	3	2	1	1	3	7	3	2	3	4	7	1	1	4	7	4	2				
0	1	0	0	1	3	0	0	0	0	2	0	2	3	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	3	4	4	6	1	1	3	2	1	5				
0	0	0	0	4	0	0	0	1	0	1	0	1	0	0	1	1	0	0	0	1	0	1	1	0	1	0	1	0	0	0	0	1	1	0	2	1	1	2	1	3	4	2	0	1	5	2	6			
1	0	0	0	3	0	0	0	0	0	1	3	3	0	0	1	0	0	1	0	0	4	0	0	0	0	0	0	0	2	2	0	1	0	3	2	1	2	2	3	6	3	3	1	4	4	1	0	2		
2	0	0	0	0	1	0	0	0	0	1	1	0	0	0	0	0	0	1	2	2	0	0	1	0	1	1	1	4	0	0	0	1	0	1	0	1	0	0	1	6	3	0	1	4	0	2	0	2		
6	1	0	0	1	0	0	2	0	1	0	1	0	0	0	0	0	4	4	2	5	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0	1	0	2	0	5	0	0	2	4	1	3	3	1	1	
2	0	1	0	0	7	0	0	4	2	0	0	0	1	4	1	0	0	2	0	3	0	0	1	0	0	2	0	2	0	1	0	0	5	0	0	0	2	0	0	0	3	4	6	5	0	3	0	0		
1	0	0	0	0	0	0	0	0	0	0	0	3	1	0	0	1	1	0	5	2	1	0	1	1	0	0	0	1	1	1	1	1	1	1	2	3	0	0	2	5	0	5	0	4	0	0	2	1		
2	0	0	0	0	1	0	1	0	0	3	0	0	0	0	0	1	3	1	0	3	2	0	0	0	0	0	0	0	3	0	0	0	3	1	2	0	4	0	1	0	0	6	2	1	0	0	1	0		
0	0	0	0	0	0	2	1	0	0	2	1	0	0	0	1	3	1	0	3	0	0	3	4	0	0	0	0	0	1	4	2	1	0	0	5	4	2	1	2	3	4	5	0	3	1	1	1	0		
0	0	1	0	0	0	0	2	0	0	2	4	0	7	4	0	1	0	1	0	1	7	0	1	0	0	0	0	0	0	6	1	0	3	0	0	2	1	1	2	4	3	4	0	2	2	0	1	1	1	
0	1	0	0	1	1	0	2	0	1	0	0	1	0	0	1	0	1	0	1	0	0	3	0	1	0	0	0	0	0	2	4	4	0	0	0	0	1	2	0	1	3	2	0	2	1	0	0	5	3	2
0	0	0	0	0	0	1	1	4	4	2	0	0	0	4	0	0	4	0	0	1	0	0	0	0	7	2	0	0	0	0	0	2	0	0	1	4	0	0	5	0	0	0	0	0	2	1	4	0		
0	0	0	1	0	0	2	9	7	0	3	2	0	0	0	0	0	0	0	1	0	2	2	1	2	4	1	2	0	0	0	1	0	0	1	0	1	1	5	1	0	3	1	1	1	1	1	0			
0	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0	1	1	0	0	5	4	3	3	2	1	0	2	1	0	0	0	0	6	7	4	0	4	3	0	7	2	1	0	0	0				
1	1	1	1	1	0	0	2	0	1	0	1	0	0	0	0	0	2	0	0	1	1	0	2	3	7	7	0	0	0	1	0	1	4	0	0	2	2	1	2	3	8	1	0	4	1	0	0	0	3	
2	0	0	0	7	0	0	0	0	0	5	0	0	1	1	0	0	0	0	0	3	0	0	1	0	5	2	5	1	7	2	0	0	1	1	2	1	0	2	1	6	2	1	0	2	0	1	2	0	1	
0	0	0	0	1	1	1	0	0	3	2	9	4	0	1	0	1	0	1	0	0	4	1	0	0	0	1	0	6	4	1	1	0	0	1	1	1	4	0	2	5	9	5	7	0	0	0	1	1	0	
0	0	0	1	0	0	0	3	0	1	8	0	0	0	0	0	0	2	0	3	1	0	3	0	6	3	0	0	1	1	3	0	1	0	4	0	0	2	1	5	7	2	0	1	0	0	0	0	0		
0	1	4	1	0	0	0	0	1	6	1	5	3	0	0	0	1	0	0	0	2	0	1	2	0	0	2	0	1	0	2	4	1	1	1	1	2	0	0	0	1	5	3	8	2	0	0	1	2		
2	1	3	0	1	2	1	2	2	5	0	2	6	0	1	1	4	0	0	3	0	0	0	1	1	0	0	0	1	0	0	1	1	0	0	6	3	0	0	1	1	0	0	0	0	1	1	0	0		
1	2	1	0	2	4	0	0	0	4	5	1	1	0	1	0	1	0	1	0	1	0	2	0	1	0	2	3	2	0	0	6	2	2	6	3	3	0	3	4	1	0	2	2	0	0	0	0	0		
3	1	4	1	0	0	4	3	0	4	0	2	0	1	0	0	0	1	1	1	4	1	1	3	0	1	0	1	3	0	0	0	0	0	3	1	2	6	3	0	3	1	0	0	4	1	0	0			

discretized point pattern: $\tilde{\mathbf{x}} = [x_{ij}]_{d_1 \times d_2}$

cell counts: $x_{ij} = n(\mathbf{x} \cap B_{ij}), i = 1, \dots, d_1, j = 1, \dots, d_2$

Convolution with kernels

- input layer of the network: discretized point pattern $\tilde{\mathbf{x}} = [x_{ij}]$
- given a set of $d_1^{(1)} \times d_2^{(1)}$ kernels/filters

$$\mathbf{F}^{(1,k)} = \left[F_{ij}^{(1,k)} \right]_{d_1^{(1)} \times d_2^{(1)}}, \quad k = 1, \dots, \ell_1$$

- $\tilde{\mathbf{x}} = [x_{ij}]$ is **convolved** with each $\mathbf{F}^{(1,k)}$

$$(\tilde{\mathbf{x}} * \mathbf{F}^{(1,k)})_{ij} = \sum_{i'} \sum_{j'} x_{i+i', j+j'} F_{i'j'}^{(1,k)}$$

valid padding $i = 1, \dots, d_1 - d_1^{(1)} + 1$ and $j = 1, \dots, d_2 - d_2^{(1)} + 1$

zero padding $x_{i+i', j+j'}$ is set to be zero for $i + i' > d_1$ and $j + j' > d_2$

discretized pattern

17	13	6	4	10	7	4	4	9	16	22	37	10	40	33	21
1	11	1	4	11	3	2	6	4	2	6	9	14	35	47	12
12	4	9	6	21	2	22	6	4	8	3	9	10	18	24	16
4	1	6	10	5	9	16	12	1	11	11	14	19	28	15	5
1	3	28	12	9	4	9	7	17	9	11	6	15	10	10	18
5	8	4	4	0	2	4	15	29	26	18	7	38	16	17	8
4	6	8	39	5	6	15	19	8	25	18	11	13	47	12	4
19	11	18	22	6	6	6	11	11	8	11	21	15	14	3	5

kernel

-1	-1	-1
-1	8	-1
-1	-1	-1

*

convolution

25	-49	-36	31	-55	-38	-7	-23	-56	-58	-39	-56	84	177
-13	29	-19	118	-73	120	-19	-18	22	-46	-14	-66	-48	16
-59	-25	-16	-33	-16	57	14	-66	24	17	28	42	103	-9
-33	176	30	26	-22	3	-47	26	-52	-14	-85	-18	-78	-37
5	-76	-73	-81	-36	-45	12	106	73	31	-74	179	-34	11
-29	-48	245	-45	4	51	53	-80	71	17	-53	-65	248	-18

input: $(d_1, d_2) = (8, 16)$ kernel: $(d_1^{(1)}, d_2^{(1)}) = (3, 3)$ convolution: $(d_1 - d_1^{(1)} + 1, d_2 - d_2^{(1)} + 1) = (6, 14)$

discretized pattern

17	13	6	4	10	7	4	4	9	16	22	37	10	40	33	21
1	11	1	4	11	3	2	6	4	2	6	9	14	35	47	12
12	4	9	6	21	2	22	6	4	8	3	9	10	18	24	16
4	1	6	10	5	9	16	12	1	11	11	14	19	28	15	5
1	3	28	12	9	4	9	7	17	9	11	6	15	10	10	18
5	8	4	4	0	2	4	15	29	26	18	7	38	16	17	8
4	6	8	39	5	6	15	19	8	25	18	11	13	47	12	4
19	11	18	22	6	6	6	11	11	8	11	21	15	14	3	5

kernel

0	1	0	0	-1	0
2	0	0	0	0	-2
0	1	0	0	-1	0

*

convolution

-18	24	-26	21	14	-4	-29	-49	-58	-123	-23
16	-41	2	2	33	3	27	-26	-48	-78	-65
-33	1	-25	35	-27	10	3	-8	-62	-22	-7
-2	-13	36	-32	-19	-31	4	-7	-6	-17	-25
1	34	5	-62	-67	-42	-9	-38	32	-10	15
9	-4	-6	42	-72	-48	5	24	-97	25	36

input: $(d_1, d_2) = (8, 16)$ kernel: $(d_1^{(1)}, d_2^{(1)}) = (3, 6)$ convolution: $(d_1 - d_1^{(1)} + 1, d_2 - d_2^{(1)} + 1) = (6, 11)$

First layer of feature maps

- convolution image: a first-order summary statistic

$$(\tilde{\mathbf{x}} * \mathbf{F}^{(1,k)})_{ij} = \sum_{\mathbf{u} \in \mathbf{x}} A_{ij}^{(k)}(\mathbf{u})$$
$$A_{ij}^{(k)}(\mathbf{u}) = \sum_{i'} \sum_{j'} \mathbb{1}[\mathbf{u} \in B_{i+i', j+j'}] F_{i'j'}^{(1,k)}$$

- output: **feature map**

$$H_{ij}^{(1,k)} = f_1 \left(b_0^{(1,k)} + (\tilde{\mathbf{x}} * \mathbf{F}^{(1,k)})_{ij} \right) = f_1 \left(b_0^{(1,k)} + \sum_{\mathbf{u} \in \mathbf{x}} A_{ij}^{(k)}(\mathbf{u}) \right)$$

- ▶ intercept (bias) term $b_0^{(1,k)} \in \mathbb{R}$
- ▶ activation function $f_1 : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ allows for non-linearity
- first layer of feature maps

$$\mathbf{H}^{(1,k)} = \left[H_{ij}^{(1,k)} \right]_{(d_1-d_1^{(1)}+1) \times (d_2-d_2^{(1)}+1)}, \quad k = 1, \dots, \ell_1$$

discretized pattern

17	13	6	4	10	7	4	4	9	16	22	37	10	40	33	21
1	11	1	4	11	3	2	6	4	2	6	9	14	35	47	12
12	4	9	6	21	2	22	6	4	8	3	9	10	18	24	16
4	1	6	10	5	9	16	12	1	11	11	14	19	28	15	5
1	3	28	12	9	4	9	7	17	9	11	6	15	10	10	18
5	8	4	4	0	2	4	15	29	26	18	7	38	16	17	8
4	6	8	39	5	6	15	19	8	25	18	11	13	47	12	4
19	11	18	22	6	6	6	11	11	8	11	21	15	14	3	5

kernel

-1	-1	-1
-1	8	-1
-1	-1	-1

convolution

25	-49	-36	31	-55	-38	-7	-23	-56	-58	-39	-56	84	177
-13	29	-19	118	-73	120	-19	-18	22	-46	-14	-66	-48	16
-59	-25	-16	-33	-16	57	14	-66	24	17	28	42	103	-9
-33	176	30	26	-22	3	-47	26	-52	-14	-85	-18	-78	-37
5	-76	-73	-81	-36	-45	12	106	73	31	-74	179	-34	11
-29	-48	245	-45	4	51	53	-80	71	17	-53	-65	248	-18

*

=

convolution

25	-49	-36	31	-55	-38	-7	-23	-56	-58	-39	-56	84	177
-13	29	-19	118	-73	120	-19	-18	22	-46	-14	-66	-48	16
-59	-25	-16	-33	-16	57	14	-66	24	17	28	42	103	-9
-33	176	30	26	-22	3	-47	26	-52	-14	-85	-18	-78	-37
5	-76	-73	-81	-36	-45	12	106	73	31	-74	179	-34	11
-29	-48	245	-45	4	51	53	-80	71	17	-53	-65	248	-18

bias

+

5

=

convolution + bias

30	-44	-31	36	-50	-33	-2	-18	-51	-53	-34	-51	89	182
-8	34	-14	123	-68	125	-14	-13	27	-41	-9	-61	-43	21
-54	-20	-11	-28	-11	62	19	-61	29	22	33	47	108	-4
-28	181	35	31	-17	8	-42	31	-47	-9	-80	-13	-73	-32
10	-71	-68	-76	-31	-40	17	111	78	36	-69	184	-29	16
-24	-43	250	-40	9	56	58	-75	76	22	-48	-60	253	-13

convolution + bias

30	-44	-31	36	-50	-33	-2	-18	-51	-53	-34	-51	89	182
-8	34	-14	123	-68	125	-14	-13	27	-41	-9	-61	-43	21
-54	-20	-11	-28	-11	62	19	-61	29	22	33	47	108	-4
-28	181	35	31	-17	8	-42	31	-47	-9	-80	-13	-73	-32
10	-71	-68	-76	-31	-40	17	111	78	36	-69	184	-29	16
-24	-43	250	-40	9	56	58	-75	76	22	-48	-60	253	-13

activation



Relu

=

feature map

30	0	0	36	0	0	0	0	0	0	0	0	89	182
0	34	0	123	0	125	0	0	27	0	0	0	0	21
0	0	0	0	0	62	19	0	29	22	33	47	108	0
0	181	35	31	0	8	0	31	0	0	0	0	0	0
10	0	0	0	0	0	17	111	78	36	0	184	0	16
0	0	250	0	9	56	58	0	76	22	0	0	253	0

First layer of feature maps

- each element of matrices $\mathbf{H}^{(1,1)}, \dots, \mathbf{H}^{(1,\ell_1)}$ contains some information about f_X

$$\begin{aligned}\mathbb{P}\left\{H_{ij}^{(1,k)} \leq z\right\} &= \mathbb{P}\left\{f_1\left(b_0^{(1,k)} + \sum_{\mathbf{u} \in X} A_{ij}^{(k)}(\mathbf{u})\right) \leq z\right\} \\ &= \sum_{n=0}^{\infty} \frac{\exp(-|W|)}{n!} \int_W \cdots \int_W \\ &\quad \mathbb{1}\left[f_1\left(b_0^{(1,k)} + \sum_{l=1}^n A_{ij}^{(k)}(\mathbf{u}_l)\right) \leq z\right] \\ &\quad f_X(\{\mathbf{u}_1, \dots, \mathbf{u}_n\}) d\mathbf{u}_1 \cdots d\mathbf{u}_n\end{aligned}$$

- transforming point patterns to feature maps

$$\mathbf{x} \xrightarrow{\text{discretization}} \tilde{\mathbf{x}} \xrightarrow{\text{feature maps}} (\mathbf{H}^{(1,1)}, \dots, \mathbf{H}^{(1,\ell_1)})$$

Pooling

- partition $\mathbf{H}^{(1,k)}$ into $p_1^{(1)} \times p_2^{(1)}$ block submatrices
- summarize entries of each submatrix into a single value by a pooling function (mean, sum, maximum)

$$\mathbf{H}^{(1,k)} \xrightarrow{\text{pooling}} \tilde{\mathbf{H}}^{(1,k)}$$

$$\tilde{\mathbf{H}}^{(1,k)} = \left[\tilde{H}_{ij}^{(1,k)} \right]_{\tilde{d}_1 \times \tilde{d}_2}$$

$$\tilde{d}_i^{(1)} = \lfloor (d_i - d_i^{(1)} + 1) / p_i^{(1)} \rfloor, \quad i = 1, 2$$

- ▶ $\tilde{\mathbf{H}}^{(1,k)}$ more **robust** to small variations in $\tilde{\mathbf{x}}$ than $\mathbf{H}^{(1,k)}$
- ▶ reduces the effect of discretization of $\mathbf{x}$

$$\mathbf{x} \xrightarrow{\text{discretization}} \tilde{\mathbf{x}} \xrightarrow{\text{feature maps}} (\mathbf{H}^{(1,1)}, \dots, \mathbf{H}^{(1,\ell_1)}) \xrightarrow{\text{pooling}} (\tilde{\mathbf{H}}^{(1,1)}, \dots, \tilde{\mathbf{H}}^{(1,\ell_1)})$$

discretized pattern

17	13	6	4	10	7	4	4	9	16	22	37	10	40	33	21
1	11	1	4	11	3	2	6	4	2	6	9	14	35	47	12
12	4	9	6	21	2	22	6	4	8	3	9	10	18	24	16
4	1	6	10	5	9	16	12	1	11	11	14	19	28	15	5
1	3	28	12	9	4	9	7	17	9	11	6	15	10	10	18
5	8	4	4	0	2	4	15	29	26	18	7	38	16	17	8
4	6	8	39	5	6	15	19	8	25	18	11	13	47	12	4
19	11	18	22	6	6	6	11	11	8	11	21	15	14	3	5

kernel

1	0	1
0	1	0
1	0	1

*

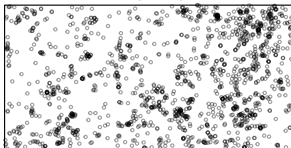
convolution

55	28	50	30	60	21	45	38	40	76	54	118	112	142
16	35	29	47	36	52	29	35	30	39	59	96	113	104
51	31	77	29	70	35	64	31	46	43	53	62	87	77
22	51	27	34	29	47	57	81	68	69	92	80	99	67
49	64	54	61	40	40	64	89	80	69	64	112	66	96
52	53	67	39	22	49	69	68	94	80	93	71	120	55

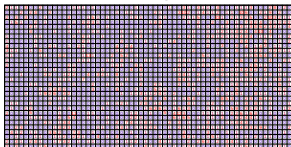
max pooling

55	50	60	45	76	118	142
51	77	70	81	69	92	99
64	67	49	89	94	112	120

observed point pattern



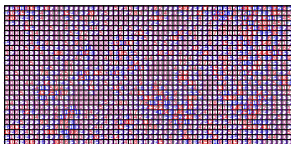
discretized pattern



kernel

0	-1	0
-1	5	-1
0	-1	0

convolution



feature map

15	20	23	9	15	5	20	5	25	5	10	10	9	13	5	14	9	10	45	26	22	117	9	13	45	25	14	38	13
25	5	14	9	5	5	10	13	12	6	10	5	5	5	7	16	5	5	9	9	9	15	19	8	34	12	17	10	10
5	5	18	9	10	5	10	12	9	10	5	9	10	18	10	5	5	15	14	9	11	12	11	14	19	22	44	35	9
5	5	9	5	14	5	10	14	7	5	5	24	22	5	10	10	10	24	5	9	14	4	20	17	21	4	19	7	8
9	5	10	29	4	23	5	8	67	5	14	23	10	10	5	15	15	9	5	15	28	5	9	17	9	19	20	18	11
5	5	10	4	5	5	13	9	5	5	9	14	10	19	9	5	5	12	9	8	12	11	15	9	28	22	7	25	14
5	10	5	12	13	7	12	5	25	18	8	48	25	18	5	5	5	10	17	14	3	20	19	12	26	18	13	14	9
10	5	9	5	19	8	10	40	12	4	10	9	10	14	5	9	5	21	18	8	5	15	14	13	9	14	13	10	23
5	10	4	47	30	15	12	5	18	5	5	24	9	18	4	25	8	14	5	10	5	18	5	23	5	19	8	8	12
9	5	8	5	14	9	5	5	5	5	10	13	8	12	18	18	29	8	22	14	7	13	38	14	15	18	32	13	5
9	8	27	5	5	9	56	5	9	10	5	5	6	12	30	27	29	51	15	5	9	7	6	42	30	26	7	9	9
5	10	8	14	8	13	22	5	9	10	9	14	35	9	16	5	14	30	26	10	17	8	19	12	15	14	5	10	5
24	16	9	5	23	14	35	11	10	9	5	18	10	8	10	13	8	11	14	9	17	8	10	8	17	15	46	5	10
15	7	21	8	4	35	11	8	9	25	10	19	9	13	6	20	9	5	25	9	30	28	13	20	14	15	9	9	5

pooled feature map

25	23	15	20	25	10	13	16	10	45	117	19	45	38	0
5	18	14	14	10	24	22	10	24	14	14	20	22	44	0
9	29	23	13	67	23	19	15	15	15	28	17	28	25	0
10	12	19	40	25	48	25	9	21	18	20	19	26	14	0
10	47	30	12	18	24	18	25	29	22	18	38	19	32	0
10	27	13	56	10	14	35	30	51	26	17	42	30	10	0
24	21	35	35	25	19	13	20	11	25	30	20	17	46	0

More layers

- second layer of feature maps $\mathbf{H}^{(2,k')} = [H_{ij}^{(2,k')}]$

$$H_{ij}^{(2,k')} = f_2 \left(b_0^{(2,k')} + \sum_k (\tilde{\mathbf{H}}^{(1,k)} * \mathbf{F}^{(2,k,k')})_{ij} \right), \quad k' = 1, \dots, \ell_2$$

► a second layer set of $d_1^{(2)} \times d_2^{(2)}$ kernels $\mathbf{F}^{(2,k,k')}$

- $\mathbf{H}^{(2,1)}, \dots, \mathbf{H}^{(2,\ell_2)}$ contain some more detailed information about f_X

$$\begin{aligned} \mathbb{P} \{ H_{ij}^{(2,k')} \leq z \} &= \sum_{n=0}^{\infty} \frac{\exp(-|W|)}{n!} \int_W \cdots \int_W \\ &\quad \mathbb{1} \left[f_2 \left(b_0^{(2,k')} + \sum_k \sum_{i'} \sum_{j'} f_1 \left(b_0^{(1,k)} + \sum_{l=1}^n A_{i+i'-1, j+j'-1}^{(k)}(\mathbf{u}_l) \right) F_{i'j'}^{(2,k,k')} \right) \leq z \right] \\ &\quad f_X(\{\mathbf{u}_1, \dots, \mathbf{u}_n\}) d\mathbf{u}_1 \cdots d\mathbf{u}_n \end{aligned}$$

- $p_1^{(2)} \times p_2^{(2)}$ pooling

$$(\tilde{\mathbf{H}}^{(1,1)}, \dots, \tilde{\mathbf{H}}^{(1,\ell_1)}) \xrightarrow{\text{feature maps}} (\mathbf{H}^{(2,1)}, \dots, \mathbf{H}^{(2,\ell_2)}) \xrightarrow{\text{pooling}} (\tilde{\mathbf{H}}^{(2,1)}, \dots, \tilde{\mathbf{H}}^{(2,\ell_2)})$$

More layers

- deeper feature maps: more layers
 - ▶ convolution with kernels
 - ▶ adding biases and applying activation functions
 - ▶ performing pooling
- number of layers: $L \geq 2$
- given $(\mathbf{H}^{(L-1,1)}, \dots, \mathbf{H}^{(L-1,\ell_{L-1})})$
- final feature vector $\mathbf{G} = [g_{k'}]$: a **perceptron** layer instead of pooling

$$g_{k'} = f_L \left(b_0^{(L,k')} + \sum_{i,j,k} H_{ij}^{(L-1,k)} w_{ij}^{(k,k')} \right), \quad k' = 1, \dots, \ell_L$$

- ▶ $b_0^{(L,k')}$ and $\mathbf{w}^{(k,k')} = [w_{ij}^{(k,k')}]$ are biases and weights

$$\mathbf{x} \xrightarrow{\text{network}} \mathbf{G} = (g_1, \dots, g_{\ell_L})$$

Convolutional neural network

- vector of all network parameters

$$\boldsymbol{\vartheta} = \left(\{b_0^{l,k}\}_{l,k}, \{F_{ij}^{l,k,k'}\}_{l,k,k',i,j}, \{w_{ij}^{k,k'}\}_{k,k',i,j} \right)$$

- for every $x \in \mathbb{R}$, $0 \leq f_L(x) \leq 1$
- a very flexible and versatile summary statistic

$$G_{\boldsymbol{\vartheta}} : \mathcal{X} \rightarrow \mathcal{F} = [0, 1]^{\ell_L}$$

- transforms any observed point pattern $\mathbf{x}$ to its corresponding final feature vector $\mathbf{G}$

$$\mathbf{x} \rightarrow \tilde{\mathbf{x}} \rightarrow \left(\begin{array}{c} \text{pooling} \\ (\mathbf{H}^{(1,1)}, \dots, \mathbf{H}^{(1,\ell_1)}) \rightarrow (\tilde{\mathbf{H}}^{(1,1)}, \dots, \tilde{\mathbf{H}}^{(1,\ell_1)}) \\ \text{pooling} \\ (\mathbf{H}^{(2,1)}, \dots, \mathbf{H}^{(2,\ell_2)}) \rightarrow (\tilde{\mathbf{H}}^{(2,1)}, \dots, \tilde{\mathbf{H}}^{(2,\ell_2)}) \\ \vdots \\ \dots \rightarrow (\mathbf{H}^{(L-1,1)}, \dots, \mathbf{H}^{(L-1,\ell_{L-1})}) \end{array} \right) \xrightarrow{G_{\boldsymbol{\vartheta}}} \mathbf{G}$$

Discriminant model

- response

$$y(\mathbf{x}^{(s_1, t_1)}, \mathbf{x}^{(s_2, t_2)}) = \begin{cases} 1 & s_1 = s_2 \\ 0 & s_1 \neq s_2 \end{cases}$$

- $y(\mathbf{x}^{(s_1, t_1)}, \mathbf{x}^{(s_2, t_2)})$: an observation of a Bernoulli random variable $Y(\mathbf{x}^{(s_1, t_1)}, \mathbf{x}^{(s_2, t_2)})$ with

$$p(\mathbf{x}^{(s_1, t_1)}, \mathbf{x}^{(s_2, t_2)}) = \mathbb{P}\{Y(\mathbf{x}^{(s_1, t_1)}, \mathbf{x}^{(s_2, t_2)}) = 1\}$$

- a general (nonlinear) discriminant model

$$p : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$$

- convert to a dissimilarity function

$$D(\mathbf{x}, \mathbf{x}') = 1 - p(\mathbf{x}, \mathbf{x}')$$

Discriminant analysis

- one approach: 1-NN , first nearest neighbor classification

$$\hat{s}^* = s_{\hat{i}}, \quad \hat{i} = \arg \max_{1 \leq i \leq N} p(\mathbf{x}^{(*)}, \mathbf{x}^{(s_i, t_i)})$$

- corresponding classifier

$$\hat{y}(\mathbf{x}^{(*)}, \mathbf{x}^{(s_i, t_i)}) = \begin{cases} 1 & s_i = \hat{s}^* \\ 0 & s_i \neq \hat{s}^* \end{cases}$$

- misclassification error

$$\max_{\mathbf{x} \in \mathcal{D}} \left| y(\mathbf{x}^{(*)}, \mathbf{x}) - \hat{y}(\mathbf{x}^{(*)}, \mathbf{x}) \right|$$

Siamese neural network (to obtain the discriminant model p)

- for any $\mathbf{x}, \mathbf{x}' \in \mathcal{D}$, compare their extracted features

$$\left. \begin{array}{l} \mathbf{x} \rightarrow G_{\boldsymbol{\vartheta}}(\mathbf{x}) \\ \mathbf{x}' \rightarrow G_{\boldsymbol{\vartheta}}(\mathbf{x}') \end{array} \right\} \rightarrow G_{\boldsymbol{\vartheta}}(\mathbf{x}) - G_{\boldsymbol{\vartheta}}(\mathbf{x}')$$

- discriminant model

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{x}') = f_{L+1} \left(\beta_0 + \sum_{k=1}^{\ell_L} \beta_k \left| [G_{\boldsymbol{\vartheta}}(\mathbf{x}) - G_{\boldsymbol{\vartheta}}(\mathbf{x}')]_k \right| \right),$$

- activation function

$$f_{L+1} : \mathbb{R} \rightarrow [0, 1]$$

- extended parameter vector

$$\boldsymbol{\theta} = (\boldsymbol{\vartheta}, \beta_0, \beta_1, \dots, \beta_{\ell_L})$$

Hyper parameters of the network

- dimensions of the discretization grid (d_1, d_2)

$$\mathbf{x} \rightarrow \tilde{\mathbf{x}}$$

- most efficient activation functions: difficult and problem specific
 - ▶ $f_1, \dots, f_{L-1}$: ReLU activation function $f(x) = \max(0, x)$
 - ▶ f_L and f_{L+1} : logistic activation function $f(x) = \exp(x)/(1 + \exp(x))$
- efficient architecture of the network: heuristic and problem specific
 - ▶ number of layers L
 - ▶ number of nodes at each layer $\ell_1, \dots, \ell_L$
 - ▶ pooling dimension and function for each convolutional layer $(p_1^{(1)}, p_2^{(1)}), \dots, (p_1^{(L-1)}, p_2^{(L-1)})$

Parameter estimation

- split the observed point patterns $\mathcal{D}$ into a training and a validation set

$$\mathcal{D}_{\text{train}} = \{\mathbf{x}^{(s,t)} : s = 1, \dots, m, t \in \{1, \dots, T\} \setminus \mathcal{J}^{(s)}\}$$

$$\mathcal{D}_{\text{valid}} = \{\mathbf{x}^{(s,t)} : s = 1, \dots, m, t \in \mathcal{J}^{(s)}\}$$

- ▶ $\mathcal{J}^{(s)} = \{J_1^{(s)}, \dots, J_{T_{\text{valid}}}^{(s)}\}$: a random sample from $\{1, \dots, T\}$ without replacement
- ▶ $\mathcal{D}_{\text{valid}}$ has T_{valid} replicates for each group
- ▶ $\mathcal{D}_{\text{train}}$ has $T - T_{\text{valid}}$ replicates for each group

- for example

$$\frac{T_{\text{valid}}}{T} \approx 0.3$$

- estimate θ by maximizing the Bernoulli **composite** log-likelihood

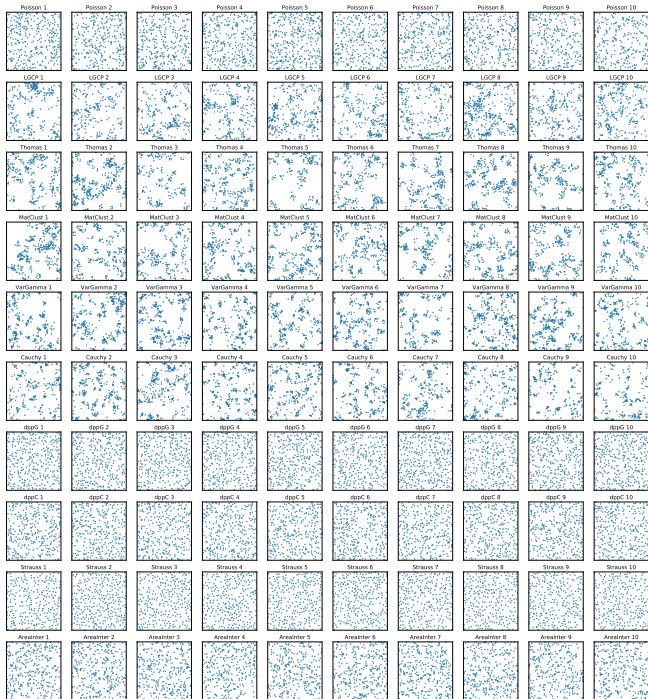
$$l(\theta; \mathcal{D}_{\text{train}}) = \sum_{\{\mathbf{x}, \mathbf{x}'\} \subset \mathcal{D}_{\text{train}}} \left\{ y(\mathbf{x}, \mathbf{x}') \log \frac{p_{\theta}(\mathbf{x}, \mathbf{x}')}{1 - p_{\theta}(\mathbf{x}, \mathbf{x}')} + \log [1 - p_{\theta}(\mathbf{x}, \mathbf{x}')] \right\}$$

Simulated point patterns

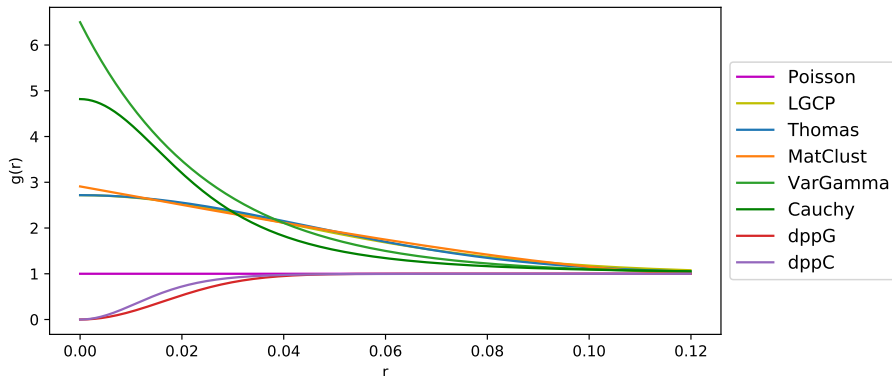
- $T = 10$ realizations from $m = 10$ stationary planar point processes
- constant intensity function $\rho(\mathbf{u}) = 300$ on $W = [0, 1]^2$
- processes
 - ▶ Poisson process
 - ▶ log-Gaussian Cox process (LGCP)
 - ★ Gaussian random field with covariance function $\exp(-(13.45r)^2)$
 - ▶ Thomas process
 - ★ parent intensity $\kappa = \frac{1}{0.004\pi(e-1)}$ and kernel variance $\omega^2 = 0.001$
 - ▶ Matérn cluster (MatClust) process
 - ★ parent intensity $\kappa = \frac{1}{0.004\pi(e-1)}$ and cluster radius $R = 0.06$
 - ▶ Variance Gamma (VarGamma) process
 - ★ parent intensity $\kappa = \frac{1}{0.004\pi(e-1)}$, shape $\nu = -1/4$ and scale $\omega = 0.025$
 - ▶ Cauchy process
 - ★ parent intensity $\kappa = \frac{1}{0.004\pi(e-1)}$ and scale $\omega = 0.015$

Simulated point patterns

- $T = 10$ realizations from $m = 10$ stationary planar point processes
- constant intensity function $\rho(\mathbf{u}) = 300$ on $W = [0, 1]^2$
- processes
 - ▶ Gaussian determinantal point process (dppG)
 - ★ scale parameter $\alpha = 0.03257$
 - ▶ Cauchy determinantal point process (dppC)
 - ★ shape parameter $\nu = 1$ and scale parameter $\alpha = 0.03257$
 - ▶ Strauss process
 - ★ interaction parameter $\gamma = 0.1$ and interaction radius $R = 0.035$
 - ▶ area-interaction (AreaInter) process
 - ★ canonical interaction parameter $\eta = 2$ and disc radius $r = 0.07$
 $R = 0.035$
 - ★ $\eta = 2 > 1$ implies weak aggregating interactions
 - ★ its realizations are similar to realizations of a Poisson process ($\eta = 1$)



Simulated point patterns



- $g(r)$ of the LGCP and Thomas processes are nearly identical
- parameter settings of dppG and dppC result in very similar kernels, their corresponding processes produce similar point patterns

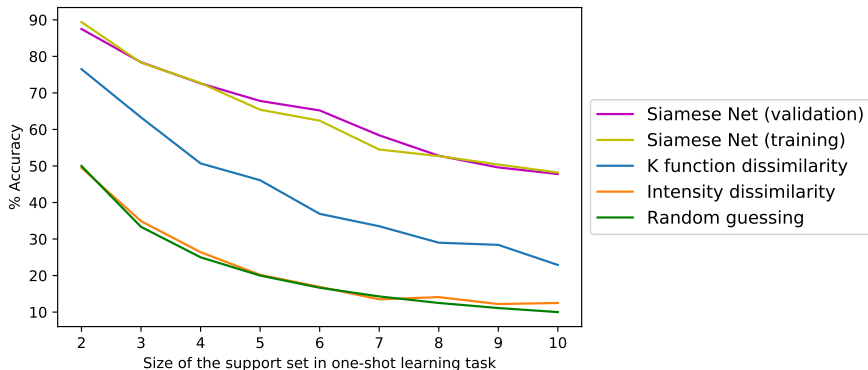
Discriminant model for the simulated point patterns

- set aside $T_{\text{valid}} = 3$ replicates for validation
- discretization $(d_1, d_2) = (128, 128)$
- network design

layer	nodes	dimensions	regularization	dropout
input:	1	$(d_1, d_2) = (128, 128)$	$\delta = 0.0025$	$p_{\text{retain}} = 0.9$
1st:	$\ell_1 = 8$	$(d_1^{(1)}, d_2^{(1)}) = (9, 9), (p_1^{(1)}, p_2^{(1)}) = (3, 3)$	$\delta = 0.0025$	$p_{\text{retain}} = 0.9$
2nd :	$\ell_2 = 16$	$(d_1^{(2)}, d_2^{(2)}) = (5, 5), (p_1^{(2)}, p_2^{(2)}) = (3, 3)$	$\delta = 0.0025$	$p_{\text{retain}} = 1$
3rd :	$\ell_3 = 32$	$(d_1^{(3)}, d_2^{(3)}) = (3, 3), (p_1^{(3)}, p_2^{(3)}) = (2, 2)$	$\delta = 0.0025$	$p_{\text{retain}} = 1$
final:	$\ell_4 = 256$		$\delta = 0.001$	$p_{\text{retain}} = 1$

- network parameter θ , with 213825 components
- estimated accuracy for $M = 2, \dots, 10$ and $n_{\text{try}} = 1000$

Discriminant model for the simulated point pattern



- intensity function dissimilarity: as good as random guessing
- K function dissimilarity: $\geq 30\%$ more accurate than random guessing
- Siamese network: $\geq 10\%$ more accurate than K function dissimilarity
 - ▶ effective in distinguishing structural differences among the processes
 - ▶ generalizable to new unobserved replicates from these processes

BCI data set

- mean number of trees per census
 - ▶ between 100 and 700 for 66 species
 - ▶ between 700 and 1500 for 32 species
 - ▶ between 1500 and 3000 for 18 species
 - ▶ between 3000 to 7000 for 8 species
 - ▶ more than 7000 for 6 species
- dissimilarity in spatial inhomogeneities of species is more definite than dissimilarity in tree to tree interactions
- ecological hypotheses: large scale variations (inhomogeneity) in diverse rainforests are more prominent in the spatial forest structure than small scale variations (tree to tree interactions)

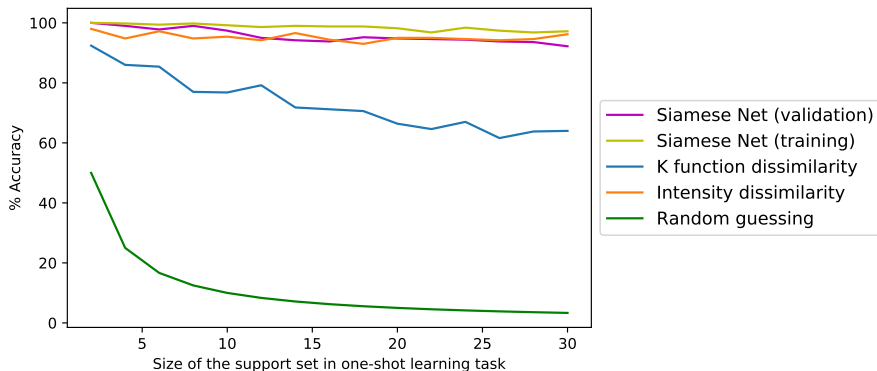
BCI data set

- $T_{\text{valid}} = 2$
- discretization $(d_1, d_2) = (256, 512)$
- network design

layer	nodes	dimensions	regularization	dropout
input:	1	$(d_1, d_2) = (256, 512)$	$\delta = 10^{-4}$	$p_{\text{retain}} = 0.9$
1st:	$\ell_1 = 64$	$(d_1^{(1)}, d_2^{(1)}) = (20, 10), (p_1^{(1)}, p_2^{(1)}) = (2, 2)$	$\delta = 10^{-4}$	$p_{\text{retain}} = 0.9$
2nd :	$\ell_2 = 128$	$(d_1^{(2)}, d_2^{(2)}) = (7, 14), (p_1^{(2)}, p_2^{(2)}) = (2, 2)$	$\delta = 10^{-4}$	$p_{\text{retain}} = 1$
3rd :	$\ell_3 = 128$	$(d_1^{(3)}, d_2^{(3)}) = (4, 8), (p_1^{(3)}, p_2^{(3)}) = (2, 2)$	$\delta = 10^{-4}$	$p_{\text{retain}} = 1$
4th :	$\ell_4 = 256$	$(d_1^{(4)}, d_2^{(4)}) = (4, 8), (p_1^{(4)}, p_2^{(4)}) = (2, 2)$	$\delta = 10^{-4}$	$p_{\text{retain}} = 1$
output:	$\ell_5 = 2048$		$\delta = 0.001$	$p_{\text{retain}} = 1$

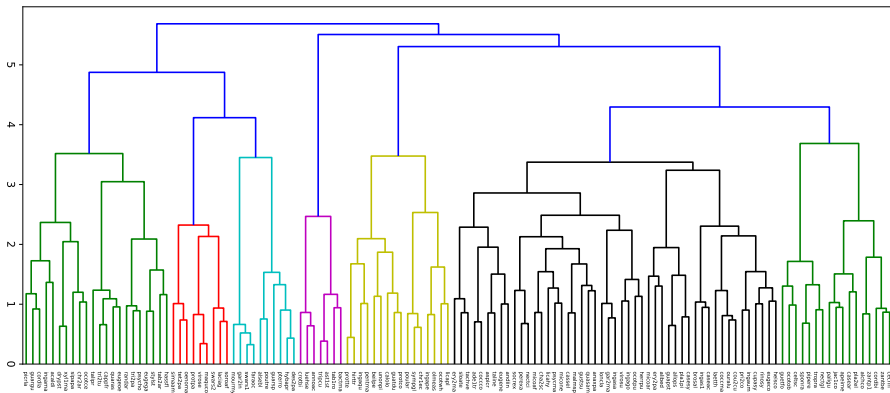
- network parameter θ , 72956449 components
- estimated accuracy for $M = 2, 4, \dots, 30$ and $n_{\text{try}} = 1000$

BCI data set



- intensity function dissimilarity: as good as Siamese network classifier
- K function dissimilarity: inferior
- better performance by tweaking the network architecture and tuning parameters

BCI data set



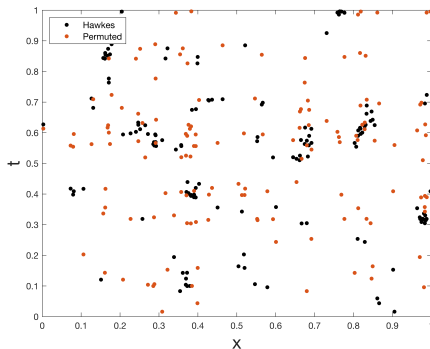
- the model distinguishes seven distinct clusters of species

2. Second order preserving permutations and data augmentation

Permutations for point processes

- Problem of testing for interactions between two spatial-temporal point processes
- In a two sample randomization test, the event times of a point process are permuted, while the spatial coordinates of the process are fixed
- This “permuted point process” provides a counterfactual that can be compared to the second point process for which an interaction is being assessed
- However, the permuted point process may not have the same statistics as the original data

Permutations for point processes



Simulated Hawkes process in black (first spatial coordinate vs. time) along with the same Hawkes process with times randomly permuted in red. Whereas the Hawkes process exhibits significant space-time clustering, the permuted process only exhibits spatial clustering.

Permutations for point processes

- **Goal:** generate permutations of a point process that preserve second order statistics of the process
- Let $\mathbf{z}_i = (\mathbf{x}_i, t_i)_{i=1}^N$ denote the spatial coordinates and event times of a point process and $\tilde{t}_i$ be a random permutation of the event times.
- Second order statistics

$$K(r) = \frac{1}{N^2} \sum_{i,j} 1\{\|\mathbf{z}_i - \mathbf{z}_j\| < r\} \quad (1)$$

and the L function, given by $L(r) = [K(r)]^{1/2}$.

- We generate permutations such that the L function, $L(r)$, of the data better matches the L function, $\tilde{L}(r)$, of the permuted point process
- **Goal:** is to generate multiple permuted point processes such that the distribution of the L functions closely matches the distribution of the L function of the data.

Permutations for point processes

- The algorithm proceeds in two stages.
- **First stage**, M independent random permutations $\tilde{\mathbf{z}}^k = (\mathbf{x}_i, \tilde{t}_i^k)$, $k = 1, \dots, M$ of the data are generated and the L function $L^k(r)$ is computed for each permutation. Next, the mean over the M L functions is calculated,

$$\mu(r) = \frac{1}{M} \sum_{k=1}^M L^k(r), \quad (2)$$

along with the error,

$$\epsilon^k(r) = L^k(r) - \mu(r). \quad (3)$$

Permutations for point processes

- **Second stage**, we generate M second order preserving (SOP) permutations. For each k , we initialize the permutation with the random permutation $\tilde{\mathbf{z}}^k$ from stage one.
- We then iteratively swap two random times to generate a proposal permutation $\tilde{\mathbf{q}}^k$. The L function, $L^{prop}(r)$, of this proposal permutation is then computed, along with the proposal error,

$$error_{prop} = \left(\int |L^{prop}(r) - L^{data}(r) - \epsilon^k(r)|^2 dr \right)^{1/2}. \quad (4)$$

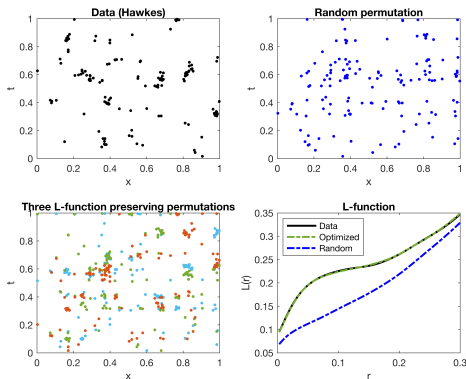
Permutations for point processes

- Generate data from a self-exciting Hawkes point process with intensity,

$$\lambda(x, t) = \mu + \sum_{t > t_i} \theta f(t - t_i; \omega) g(x - x_i; \sigma). \quad (5)$$

- We simulate the process on the unit cube in space-time with background rate $\mu = 40$, reproduction number $\theta = 0.75$, exponential kernel f in time with parameter $\omega = 100$ (mean 0.01) and Gaussian kernel g in space (2d) with standard deviation $\sigma = 0.01$.

Permutations for point processes



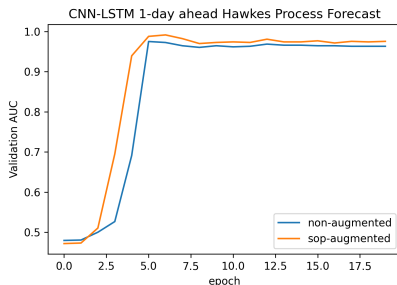
Top Left: Realization of the Hawkes process (first spatial coordinate vs. time). See text for the parameters used. Top right: example random permutation. Lower left: Three example SOP permutations of the Hawkes process. Lower Right: L function of data vs. L functions of SOP and random permutations.

Data augmentation for CNN-LSTM 1-day ahead point process forecast

- A number of neural network based models for space-time point processes have been introduced recently
- These models can improve accuracy of forecasts over simpler parametric models, but typically require larger datasets due to the increased number of parameters.
- We show how SOP permutations can be used to improve model performance on held out data through augmentation of training data.
- We generate data from a space-time Hawkes process defined on $[0, 1] \times [0, 1] \times [0, 730]$. Next we discretize space into 25×25 grid cells and time into 1 “day” intervals from 0 to 730. We then use a sliding window and create $14 \times 25 \times 25$ features consisting of a binary indicator in each space-time cell for whether at least one event occurred ($y = 1$) or no events occurred ($y = 0$).

Data augmentation for CNN-LSTM 1-day ahead point process forecast

- Use this feature as input to a CNN-LSTM to predict whether an event will occur or not in each grid cell in the following day.
- First we train the model with no data augmentation.
- We then use SOP permutations to increase the data set size by a factor of 10.



AUC on held-out data versus training epochs for a CNN-LSTM forecast using sop-augmented training and non-augmented training.

3. Spatio-temporal-network point processes for modeling crime incidents with landmarks

Self-exciting and crime data

- Self-exciting point processes have been used in crime modeling with successful attempts on burglary, gang violence, and terrorism data
- They are well-suited to characterize both the *endogenous* crime rates and the *near-repeat pattern* observed in crime data, thanks to the statistical structure proposed in **Hawkes processes**
- Self-exciting point processes model the event intensity using a background event rate and a so-called *influence kernel*, which plays a pivotal role in capturing the triggering effect of an observed crime event to the likelihood of more event occurrences in the future
- The contagion of crime, especially within the urban environment, is complex and well influenced by the city's geography

Crime data in Valencia, Spain

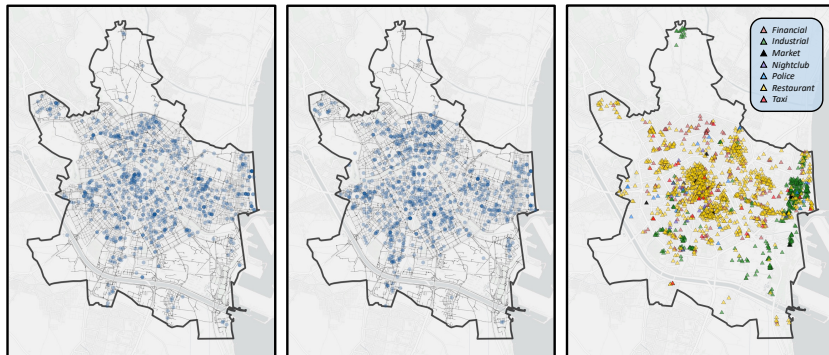


Figure: (a, b) Two snapshots of crime incidents in July 2016 and October 2018 in the city of Valencia (Spain). The blue dots represent the recorded incidents by the local police department, with a deeper color indicating multiple incidents within a small area. The grey lines represent the street network in the city of Valencia. (c) Locations of city landmarks. Each triangle represents one landmark, with different colors suggesting different landmark types.

Data: some facts

- Records thefts and robberies that happened during a five-year period from 2015 to 2019, including a total of 47,125 crime events
- The recorded events are categorized into three distinct types, including: (i) *Assault*, referring to thefts involving physical assault, (ii) *Subtraction*, referring to thefts executed smoothly without the use of force, and (iii) *Others* referring to other types of street thefts or robberies not fitting into the previous categories
- We also collect the location information of 1,975 city landmarks in Valencia. A total of seven landmark categories are collected, including financial, industrial, market, nightclub, police, restaurant, and taxi

Notation

- Sequence of n reported crime incidents in Valencia

$$(t_1, s_1, c_1), (t_2, s_2, c_2), \dots, (t_n, s_n, c_n)$$

- For the i -th event, $t_i \in [0, T]$ represents the time of incident occurrence, $s_i \in \mathcal{S} \subseteq \mathbb{R}^2$ denotes the location of the incident, and $c_i \in C := \{1, 2, 3\}$ denotes the crime category of the i -th incident, with 1, 2, and 3 representing *Assault*, *Subtraction*, and *Others*
- All the events are temporally ordered, $0 \leq t_1 < t_2 < \dots < t_n \leq T$. The entire sequence can be denoted as $\mathcal{H}_T = \{(t_i, s_i, c_i)\}_{i=1}^n$
- We also introduce the notation for the set of landmark categories as $L := [1 : 7]$. Values from 1 to 7 correspond to the landmark type of financial, industrial, market, nightclub, police, restaurant, and taxi

Zones

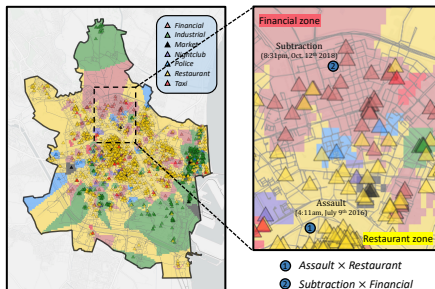


Figure: Partition of Valencia city area into various urban functional zones

- Each given location $s \in \mathcal{S}$ can be assigned with that landmark category $\ell(s)$ that is the most common among its k nearest landmarks, $\ell(s) : \mathcal{S} \rightarrow L$
- The partition of functional zones is formed by grouping locations labeled with the same landmark category, $\{\mathcal{S}_l\}_{l \in L}$

Mark events

- The mark of an event is designed to be the concatenation of its crime category c and the landmark category $\ell(s)$ determined by its location s
- The labeling function $\ell(s)$ identifies the urban functional zone. We denote the event mark as $c \times \ell(s)$
- The event mark derives a detailed categorization of crime events by including additional event features that go beyond the time or location of the observed event. The resulting mark space is a finite set $C \times L$ with a size of 21 (three crime categories and seven landmark categories).

Interaction network

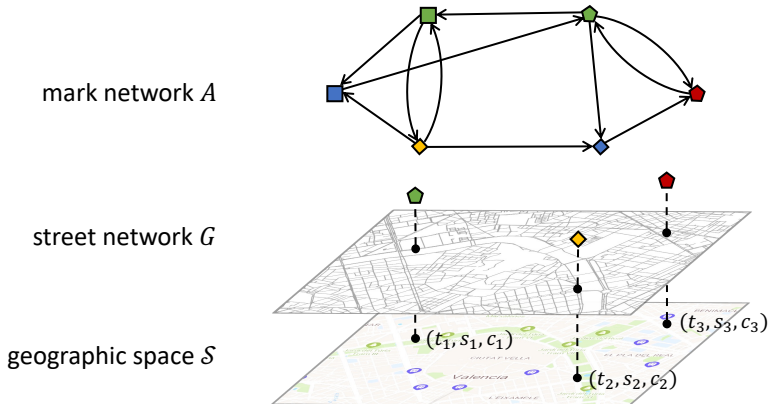


Figure: Multiple spaces for event dependence. An overlay of street network G on top of the two-dimensional geographic space $\mathcal{S}$. The crime events (black dots) can be mapped onto the corresponding edges of network G according to their locations in the space $\mathcal{S}$. Another network A captures the event dependence over the mark space (marks represented by various shapes and colors)

Interaction network

- These endpoints are treated as the nodes of the street network, and two nodes are connected by a network edge if there is a street segment between them
- Each network edge has an attribute, known as the *edge weight*, indicating the length of the corresponding street segment measured in kilometers
- The entire network is processed to be undirected
- The final street network structure consists of 8,043 nodes and 12,309 weighted, undirected edges, covering the entire city area of Valencia
- The mark network is constructed with nodes representing distinct crime-landmark labels, and the events are assigned to different nodes of the network $\mathcal{A}$ according to their crime-landmark labels
- The edges of the network indicate the potential relation between the corresponding crime-landmark labels they connect with

Multivariate spatio-temporal point processes on a network

- Let $\mathcal{H}_t = \{(t_i, s_i, c_i \times l_i) \in \mathcal{H}_T | t_i < t\}$ denote the observed history of crime events happened before time t
- We adopt a *conditional intensity function* for each event category $c \times l$ to suggest the possibility of observing a new incident with label $c \times l$ at time t and location s conditioning on the history $\mathcal{H}_t$
- The conditional intensity function is defined as

$$\lambda_{cl}(t, s | \mathcal{H}_t) = \lim_{\Delta t \downarrow 0, \Delta s \downarrow 0} \frac{\mathbb{E}[\mathbb{N}_{cl}([t, t + \Delta t] \times B(s, \Delta s)) | \mathcal{H}_t]}{|B(s, \Delta s)| \Delta t}, \quad s \in \mathcal{S}_l,$$

where $B(s, \Delta s)$ is a ball centered at location s in the urban functional zone $\mathcal{S}_l$ with radius Δs

- The conditional intensity function can be interpreted as the rate of incident occurrences at time t and location s

Hawkes spatio-temporal point processes

- Hawkes processes assume that the occurrences of future events are influenced by the observed history positively, and the influence of past events is linearly additive.
- We model the conditional intensity function as follows

$$\lambda_{cl}(t, s \mid \mathcal{H}_t) = \mu_{cl} + \sum_{(t', s', c' \times l') \in \mathcal{H}_t} k(t', t, s', s, c' \times l', c \times l), \quad s \in \mathcal{S}_t. \quad (6)$$

Here μ_{cl} represents the baseline intensity of events with label $c \times l$, which can be a constant or a function of time t and location s .

- The k function is an influence kernel that captures the influence of a past incident $(t', s', c' \times l')$ on a current event $(t, s, c \times l)$.
- The influence kernel k can be expressed as

$$k(t', t, s', s, c' \times l', c \times l) = f(t', t) \cdot g(s', s) \cdot h(c' \times l', c \times l).$$

Hawkes spatio-temporal point processes

- **Temporal kernel** We choose our temporal kernel f to be an exponential function

$$f(t', t) = \beta e^{-\beta(t-t')}.$$

- **Street-network-based spatial kernel** We adopt a network-based distance measurement for accurately modeling criminals' movement patterns. We propose a Gaussian spatial kernel as

$$g(s', s) = \frac{1}{2\pi\sigma^2} e^{-\frac{d_{\text{net}}^2(s, s')}{2\sigma^2}}.$$

- **Interactions between event marks** We represent the kernel function h using a set of coefficients $\{\alpha_{cl, c'l'}\}_{c, c' \in C, l, l' \in L}$, where

$$h(c' \times l', c \times l) = \alpha_{cl, c'l'}$$

captures the influence of a historical event with mark $c' \times l'$ on a future event with mark $c \times l$

Hawkes spatio-temporal point processes

- The conditional intensity function for a crime event with mark $c \times l$ at time t and location s is modeled as follows

$$\lambda_{cl}(t, s) = \mu_{cl} + \sum_{(t', s', c' \times l') \in \mathcal{H}_t} \alpha_{cl, c' l'} \beta e^{-\beta(t-t')} \frac{e^{-\frac{d_{\text{net}}^2(s, s')}{2\sigma^2}}}{2\pi\sigma^2}, \quad s \in \mathcal{S}_l. \quad (7)$$

- The base intensity μ_{cl} is set as a constant to be estimated from the data
- The influence kernel is chosen to integrate to $\alpha_{cl, c' l'}$, providing a natural interpretation of the coefficient: the expected number of crime events with mark $c \times l$ triggered by an observed event with mark $c' \times l'$

Influence kernel learning with graph neural networks

- Coefficients $\{\alpha_{cl,c'l'}\}_{c,c' \in C, l,l' \in L}$ play a pivotal role in capturing the event dependence and indicating the underlying network structure over the mark space
- We first decompose the coefficients into two components

$$\alpha_{cl,c'l'} = a_{cl,c'l'} \cdot p_{cl,c'l'},$$

where $a_{cl,c'l'} > 0$ and $0 \leq p_{cl,c'l'} \leq 1$ are both scalars, and $\sum_{c' \in C, l' \in L} p_{cl,c'l'} = 1$

- Together these two components can be viewed as the *strength* and the *chance* of the interaction between two marks, such that
- $p_{cl,c'l'}$ can capture the graph topology and *incorporate mark features* in the sense that it gives, for any $c \times l$, the chance that it has a connection to the rest of $c' \times l'$
- the $a_{cl,c'l'}$ captures the weights on the edges, indicating the strength of the connection

Influence kernel learning with graph neural networks

- For the *chance* variables, we use Graph Attention Networks (GAT) that computes the chance of the influence from mark $c' \times l'$ to mark $c \times l$ through a so-called multihead self-attention mechanism over graphs
- The attention mechanism in the r -th attention head is

$$e_{cl,c'l'}^r = \gamma(W^r X_{cl}, W^r X_{c'l'}),$$

- $X_{cl} \in \mathbb{R}^d$ denotes the d -dimensional column vector that represents the feature of mark $c \times l$
- the weight matrix $W^r \in \mathbb{R}^{D' \times D}$ defines a shared linear transformation of each mark feature
- $\gamma: \mathbb{R}^{D'} \times \mathbb{R}^{D'} \rightarrow \mathbb{R}$ takes a pair of transformed features as input and compute the attention weight $e_{cl,c'l'}^r$

Influence kernel learning with graph neural networks

- The attention weights get normalized leading the output of the r -th head

$$\begin{aligned} p_{cl,c'l'}^r &= \text{softmax}(\{e_{cl,c'l'}^r\}_{c' \in C, l' \in L}) \\ &= \frac{\exp(\text{LeakyReLU}(b^{r\top} [W^r X_{cl} \| W^r X_{c'l'}]))}{\sum_{c' \in C, l' \in L} \exp(\text{LeakyReLU}(b^{r\top} [W^r X_{cl} \| W^r X_{c'l'}]))}. \end{aligned} \quad (8)$$

- $\text{LeakyReLU}(x) = \max(0, x) + b \min(0, x)$. We set $b = 0.2$
- The results from different attention heads are then averaged to get the final chance variable $p_{cl,c'l'}$ as $p_{cl,c'l'} = \frac{1}{R} \sum_{r=1}^R p_{cl,c'l'}^r$
- For the *strength* variable $a_{cl,c'l'}$, we treat it as a trainable scalar that is directly learned from data
- The learnable parameters are $\{b^r, W^r\}_{r=1}^R$ in GAT and the interaction strength $\{a_{cl,c'l'}\}_{c,c' \in C, l,l' \in L}$.

Model estimation

- We estimate the base intensity $\{\mu_{cl}\}_{c \in C, l \in L}$ by the average number of observed crime events with mark $c \times l$ over a street segment and time interval of unit length (i.e., one kilometer and one day)
- The influence kernel is estimated by maximizing the log-likelihood function of the point process model. The parameters in the influence kernel are $\theta := \{\{b^r, W^r\}_{r=1}^R, \{a_{cl, c'l'}\}_{c, c' \in C, l, l' \in L}, \beta, \sigma\}$
- The log-likelihood function of observing $\mathcal{H}_T = \{(t_i, s_i, c_i \times l_i)\}_{i=1}^n$ on $[0, T] \times \mathcal{S}$ is given by

$$L(\theta) = \sum_{i=1}^n \log \lambda_{c_i l_i}(t_i, s_i) - \sum_{c \in C, l \in L} \int_0^T \int_{\mathcal{S}} \lambda_{cl}(t, s) ds dt \quad (9)$$

where θ is incorporated into the conditional intensity function.

- Solving the M-step in the classic expectation-maximization (EM) algorithm for point processes becomes intractable and too expensive. We thus adopt the stochastic gradient descent technique

Results

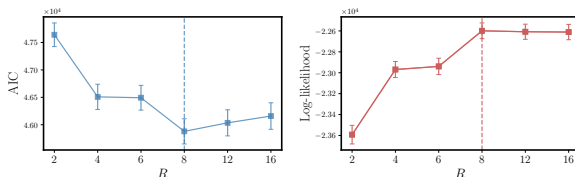


Figure: Cross validation results for choosing the optimal number of attention heads R . *Left*: AIC. *Right*: log-likelihood on the hold-out data

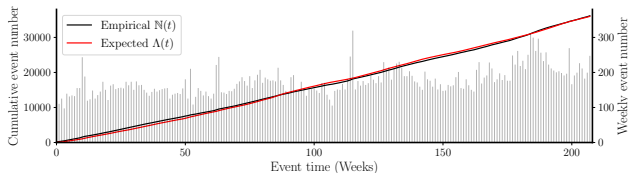


Figure: Empirical and expected cumulative number of events against the event times, represented by the black and red lines. The grey vertical lines indicate the number of crime events in each week

Results

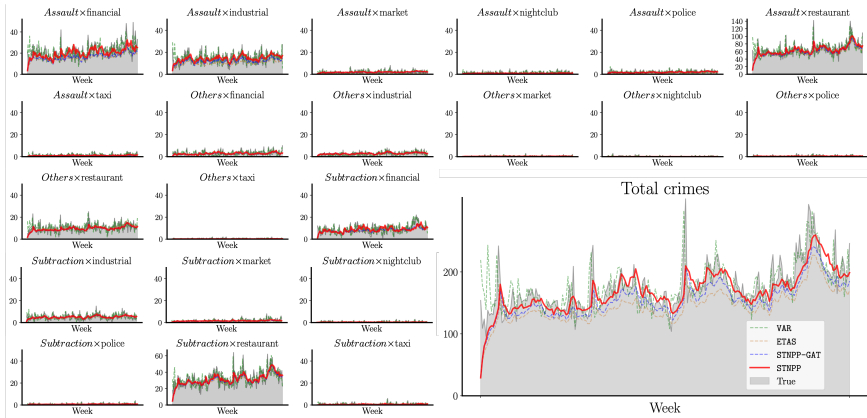


Figure: In-sample estimation of the number of crime events from 2015 to 2018 by different models. The red lines represent the in-sample estimations by our model STNPP. The dashed blue, yellow, and green lines represent the in-sample estimations by three baselines. The grey areas indicate the number of true observations

Results

Table: Quantitative results of data fitting and **in-sample estimation**. Bold indicates the best performance.

Model	MAE (rare)	MAE (frequent)	MAE (total)	Training log-likelihood	AIC
Persistent	0.998	5.736	31.538	/	/
VAR	0.906	3.680	21.940	/	/
ETAS	0.785	4.266	30.925	-2.476	45039.270
STNPP-GAT	0.728	3.875	21.561	-2.427	44173.386
STNPP	0.716	3.708	20.080	-2.413	44099.266

Table: Quantitative results of **out-of-sample estimation**. Bold indicates the best performance.

Model	MAE (rare) (↓)	MAE (frequent) (↓)	MAE (total) (↓)	Testing log-likelihood (↑)
Persistent	1.006	5.803	28.808	/
VAR	0.998	5.502	27.507	/
ETAS	0.879	4.786	28.715	-2.223
STNPP-GAT	0.769	4.329	26.302	-2.201
STNPP	0.773	4.223	21.788	-2.183

Results

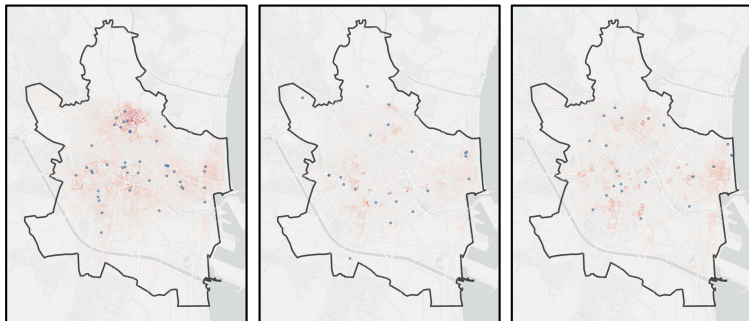


Figure: Out-of-sample prediction for the event intensity over the street network by STNPP. Each panel shows the predicted intensity of one type of crime on a given date in 2019 (Assaults, Others, Substraction). The depth of the red color indicates the value of the conditional intensity. The blue dots represent actual incidents reported in the next two days from the given date

Results

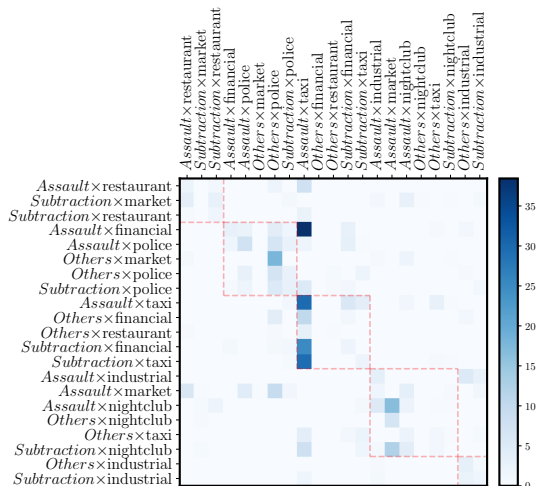
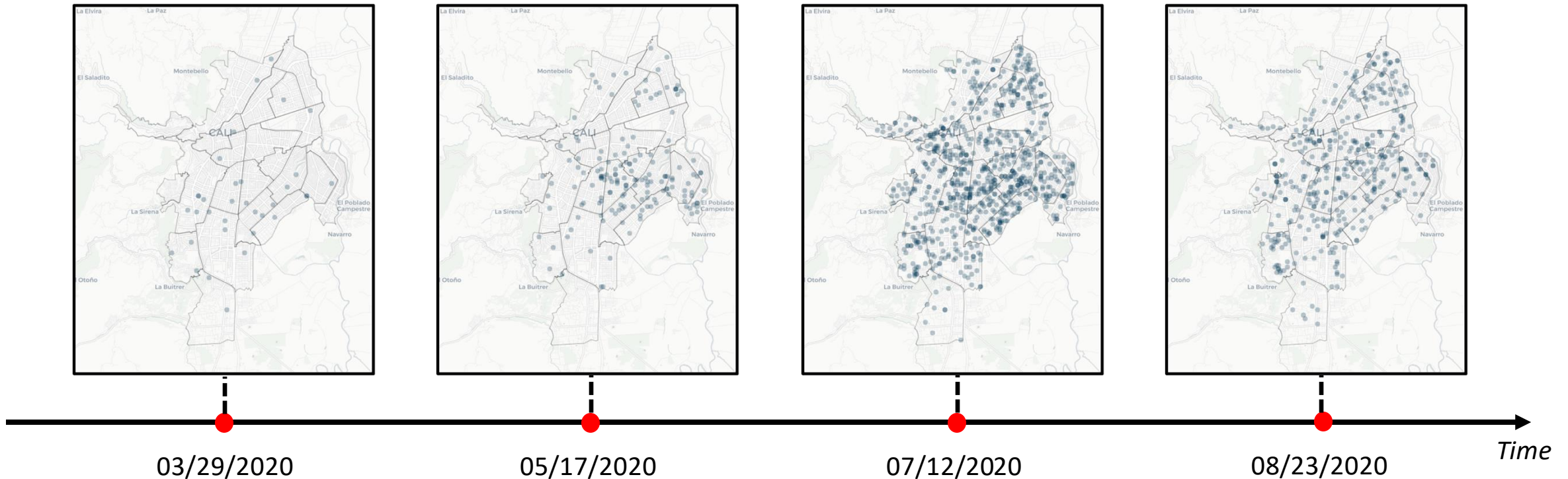


Figure: Learned coefficients by STNPP. The red dashed lines indicate communities detected on the weighted mark networks constructed from the learned coefficients

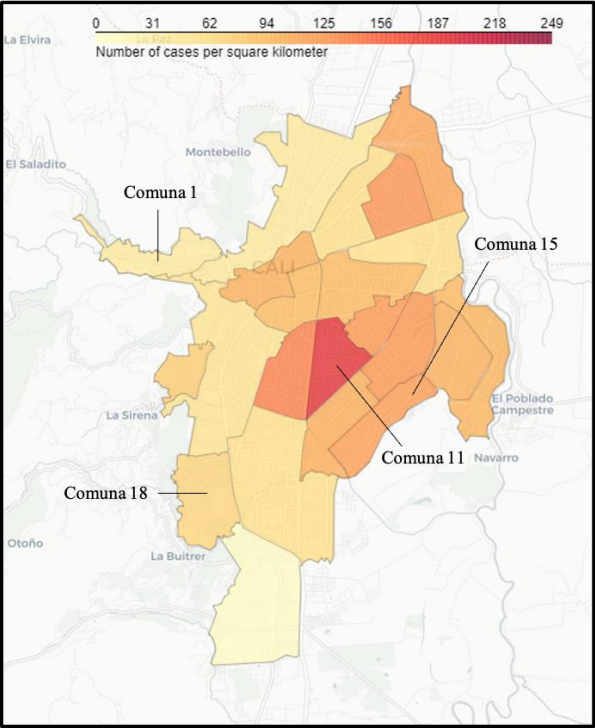
4. Non-stationary deep spatial-temporal point process modeling

Unique COVID-19 Data: discrete events

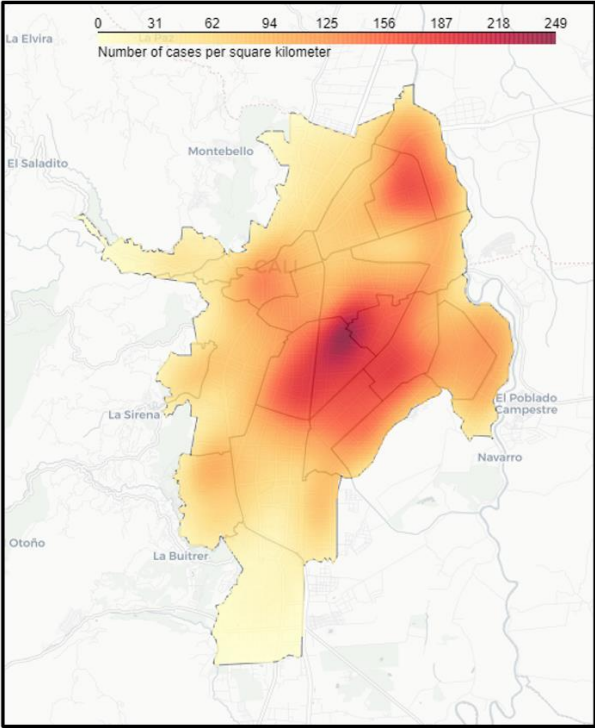
- 38,611 cases from 03/15/2020 to 09/30/2020 in Cali, Colombia
- **Temporal-resolution:** Daily basis
- **Spatial-resolution:** longitude and latitude of each case



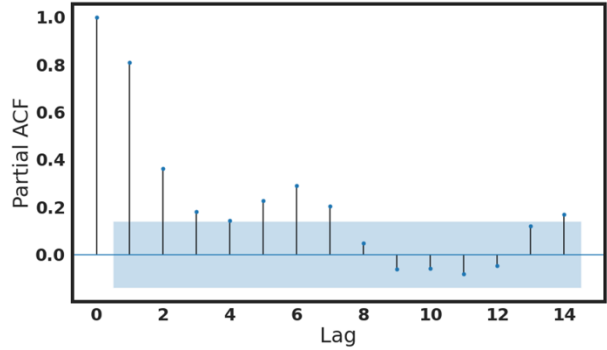
Data distribution and correlation



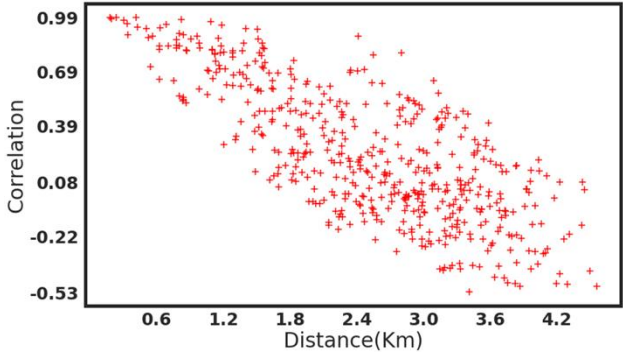
Case density by comuna



Case density by KDE



Temporal correlation



Spatial correlation

Non-stationary neural kernel

$$k(t, t', s, s') = \boxed{v(t, t')} \cdot \boxed{v(s, s')}$$

Reason: heterogenous spread patterns in different areas

Stationary temporal kernel

$$v(t, t') = C e^{-\frac{1}{2\sigma^2}(t-t')^2}$$

Non-stationary spatial kernel

$$v(s, s') = \langle \phi_s, \phi_{s'} \rangle$$

- Feature functions $\phi_s(\cdot) = \sum_{r=1}^R w_s^{(r)} \kappa_s^{(r)}(\cdot)$
- Choose $\kappa_s(\cdot)$ to be a Gaussian component centered at location s with covariance Σ_s

Non-stationary neural kernel

$$\begin{aligned}
 v(s, s') &= \langle \sum_{r=1}^R w_s^{(r)} \kappa_s^{(r)}(\cdot), \sum_{r=1}^R w_{s'}^{(r)} \kappa_{s'}^{(r)}(\cdot) \rangle \\
 &= \sum_{1 \leq r_1, r_2 \leq R} \frac{w_s^{(r_1)} w_{s'}^{(r_2)}}{2\pi \left| \Sigma_s^{(r_1)} + \Sigma_{s'}^{(r_2)} \right|^{\frac{1}{2}}} \exp \left\{ -\frac{1}{2} (s - s')^T \left(\Sigma_s^{(r_1)} + \Sigma_{s'}^{(r_2)} \right)^{-1} (s - s') \right\}
 \end{aligned}$$


Captures different influence patterns
at different locations

Non-stationary neural kernel

$$v(s, s') = \langle \sum_{r=1}^R w_s^{(r)} \kappa_s^{(r)}(\cdot), \sum_{r=1}^R w_{s'}^{(r)} \kappa_{s'}^{(r)}(\cdot) \rangle$$

$$= \sum_{1 \leq r_1, r_2 \leq R} \frac{w_s^{(r_1)} w_{s'}^{(r_2)}}{2\pi |\Sigma_s^{(r_1)} + \Sigma_{s'}^{(r_2)}|^{\frac{1}{2}}} \exp \left\{ -\frac{1}{2} (s - s')^T \left(\Sigma_s^{(r_1)} + \Sigma_{s'}^{(r_2)} \right)^{-1} (s - s') \right\}$$



How to parametrize a covariance matrix that smoothly varies over space?

One-to-one mapping between the covariance matrix and the one standard deviation ellipse of the distribution



Represent the covariance matrix using the focal points of the ellipse (fixed ellipse area A)

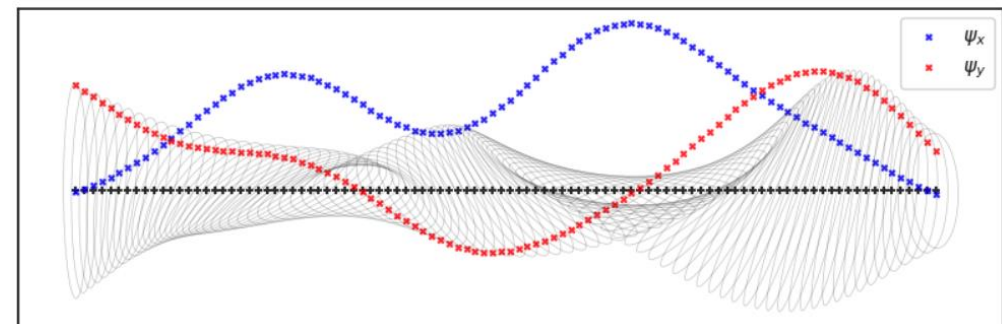


Parametrize focal points as a function of location s

focal point

$$\Sigma_s = \tau_z^2 \begin{pmatrix} Q + \|\psi_s\|^2 \cos 2\alpha / 2 & \|\psi_s\|^2 \sin 2\alpha / 2 \\ \|\psi_s\|^2 \sin 2\alpha / 2 & Q - \|\psi_s\|^2 \cos 2\alpha / 2 \end{pmatrix}$$

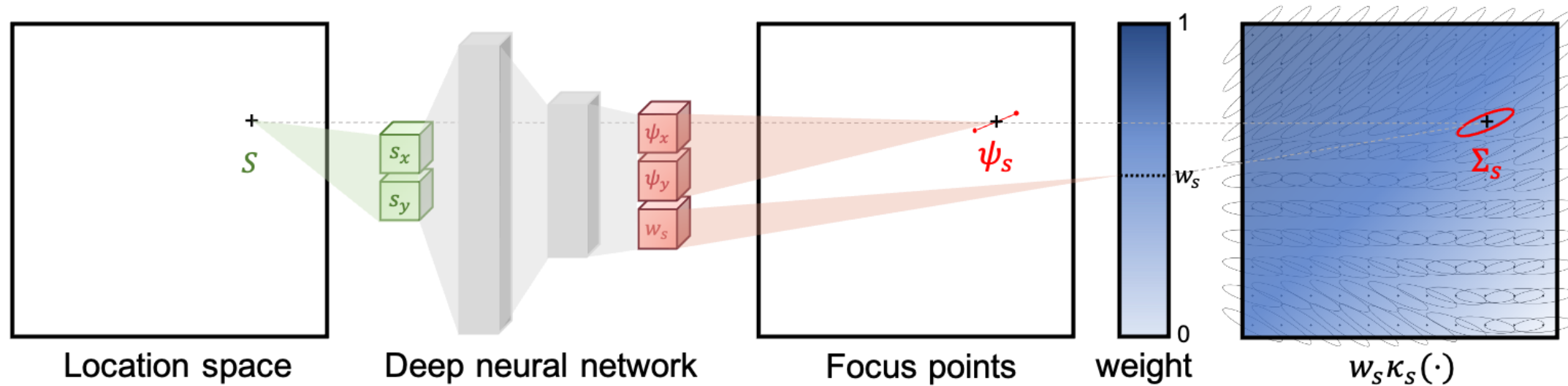
$$\begin{cases} Q = \sqrt{4A^2 + \|\psi_s\|^4 \pi^2} / (2\pi) \\ \alpha = \tan^{-1}(\psi_s^y / \psi_s^x) \\ \tau_z: \text{scaling parameter} \end{cases}$$



Location s

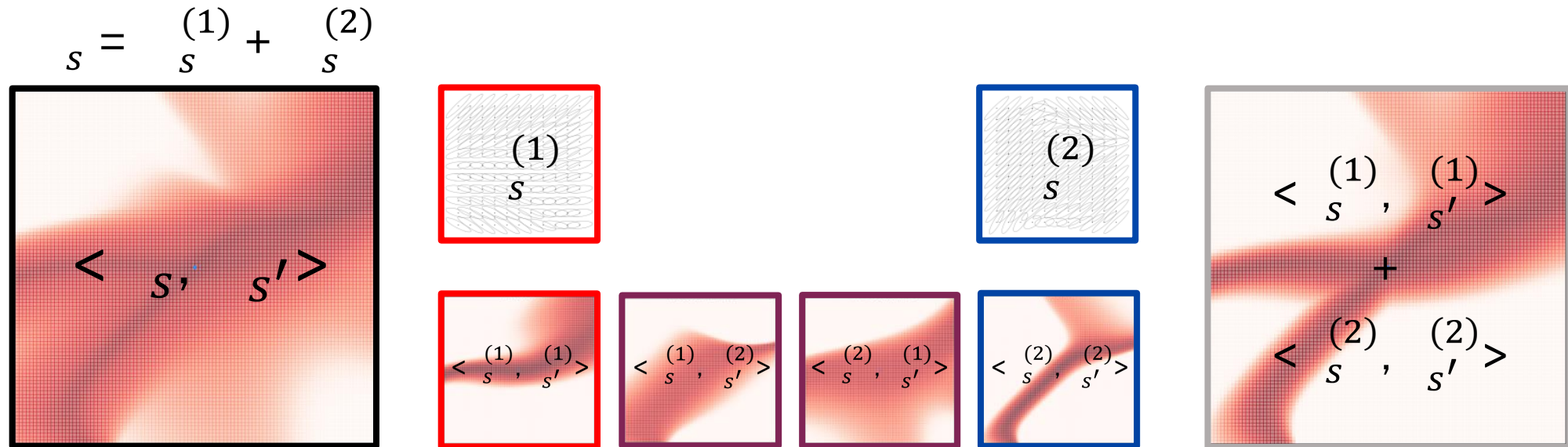
Mapping from s to Σ_s via neural networks

Represent mapping from s to ψ_s and w_s using deep neural networks



Non-stationary spatial kernel

Example of a non-stationary spatial kernel $v(s, \cdot) = \langle \phi_s, \phi \cdot \rangle$

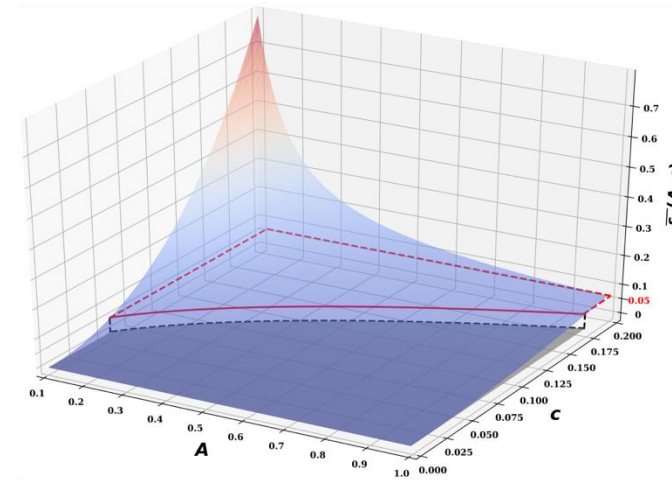


Efficient computation

$$\min_{\theta} -\ell(\theta) = -\sum_{i=1}^n \log \lambda(t_i, s_i) + \int_0^T \int_{\mathcal{S}} \lambda(t, s) ds dt$$

Challenges of computing $\int_0^T \int_{\mathcal{S}} \lambda(\tau, u) du d\tau$:

- ❑ Intractable integral due to irregular space $\mathcal{S}$ and neural kernel.
- ❑ Numerical integration is expensive of complexity $\mathcal{O}(N^3)$ (5mins/epoch). (Here $N = 38611$)



Proposition:

The double integral in the log-likelihood can be approximated by

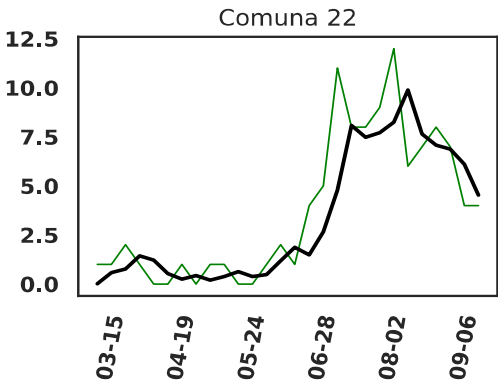
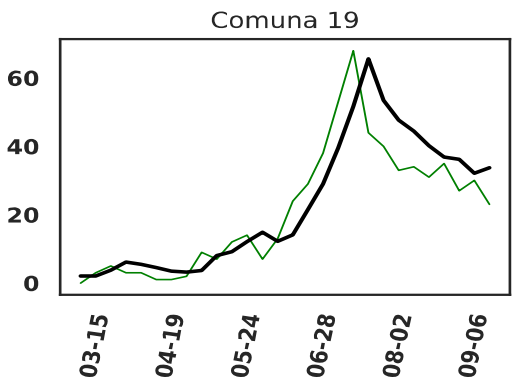
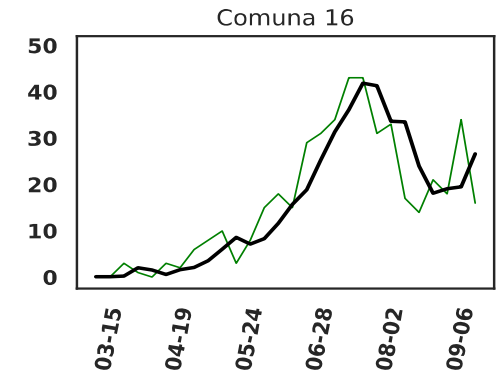
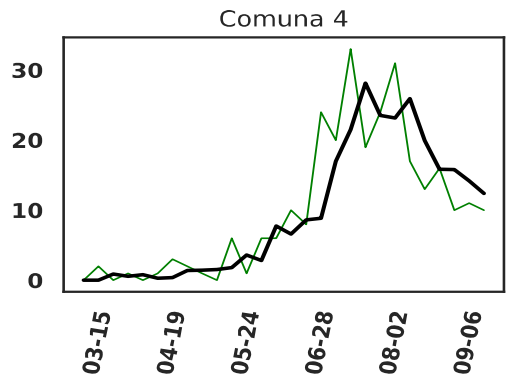
$$\int_0^T \int_{\mathcal{S}} \lambda(\tau, r) dr d\tau = (1 + \epsilon_2) \left[\lambda_0 |\mathcal{S}| T + T \sum_{l=1}^L \gamma_l + \sqrt{2\pi} C \sigma_0 \sum_{i=1}^{\mathbb{N}([0, T] \times \mathcal{S})} \left\{ h\left(\frac{T - t_i}{\sigma_0}\right) - \frac{1}{2} \right\} \right] - \epsilon_1,$$

with the error controlled by

$$|\epsilon_2| < \max \left\{ U - 1, 1 - \frac{1}{U} \right\}, U = \frac{\sqrt{4A^2 + c^4\pi^2} + c^2\pi}{2A}.$$

$\mathcal{O}(N)$ Complexity
(5seconds/epoch)

In-sample prediction



Observation

NSSTPP($R=3$)

NSSTPP-Exo($R=3$)

ETAS

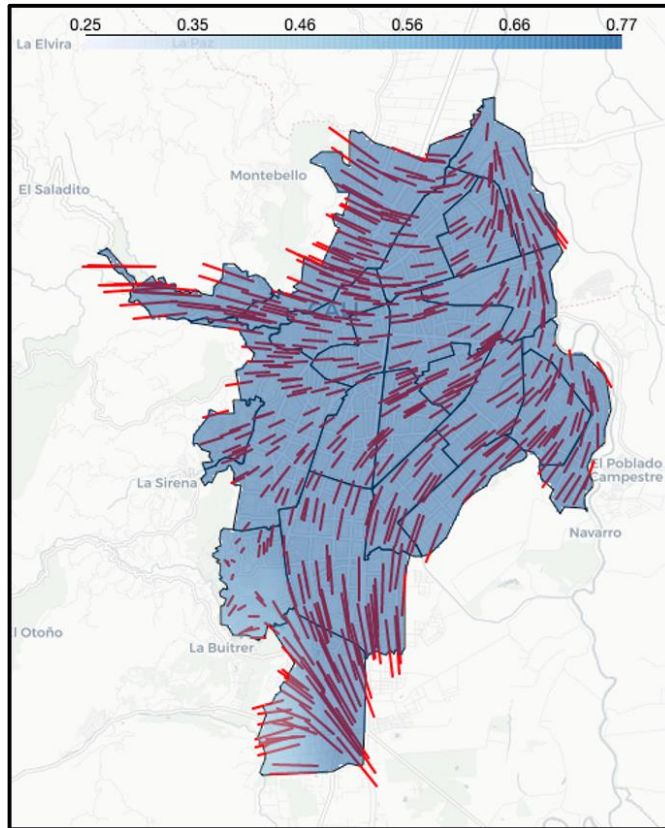
SIR

AR(3)

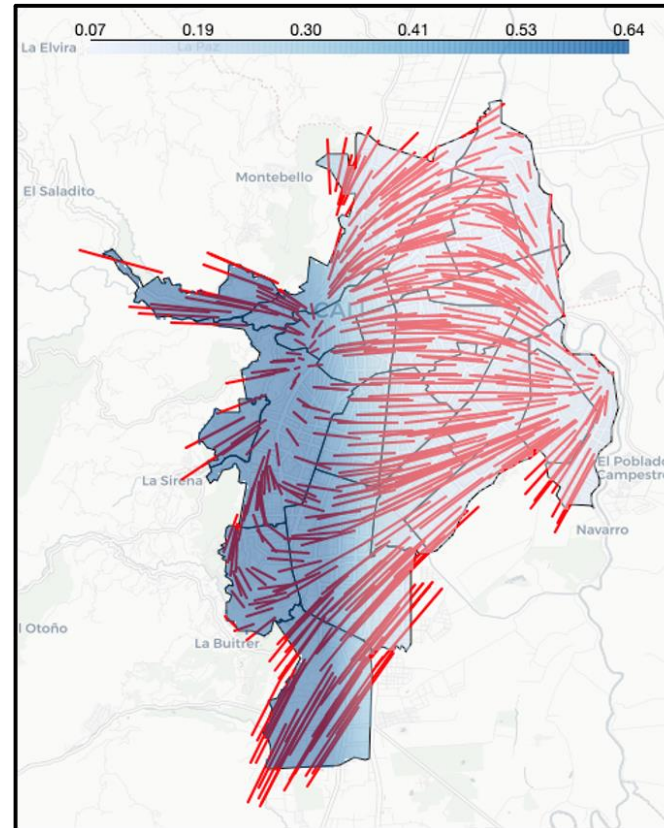
Models	Log-likelihood($\times 10^4$)	MAE $Q_{0.25}^{\text{in}}$	MAE $Q_{0.5}^{\text{in}}$	MAE $Q_{0.75}^{\text{in}}$
Random	/	5.000	11.000	18.000
SIR	/	1.862	3.759	7.391
AR(3)	/	1.307	2.880	6.496
ETAS	4.868 _(0.0058)	1.486 _(0.102)	4.737 _(0.240)	14.895 _(0.420)
NSSTPP-Exo ($R=1$)	8.671 _(0.0772)	0.834 _(0.087)	3.145 _(0.186)	7.922 _(0.374)
NSSTPP-Exo ($R=2$)	9.138 _(0.0886)	0.806 _(0.088)	2.728 _(0.178)	7.119 _(0.361)
NSSTPP-Exo ($R=3$)	9.190 _(0.0906)	0.853 _(0.090)	2.613 _(0.165)	7.000 _(0.339)
NSSTPP ($R=3$)	9.331 _(0.0937)	0.797 _(0.085)	2.620 _(0.161)	6.757 _(0.330)

Kernel interpretation – Gaussian component

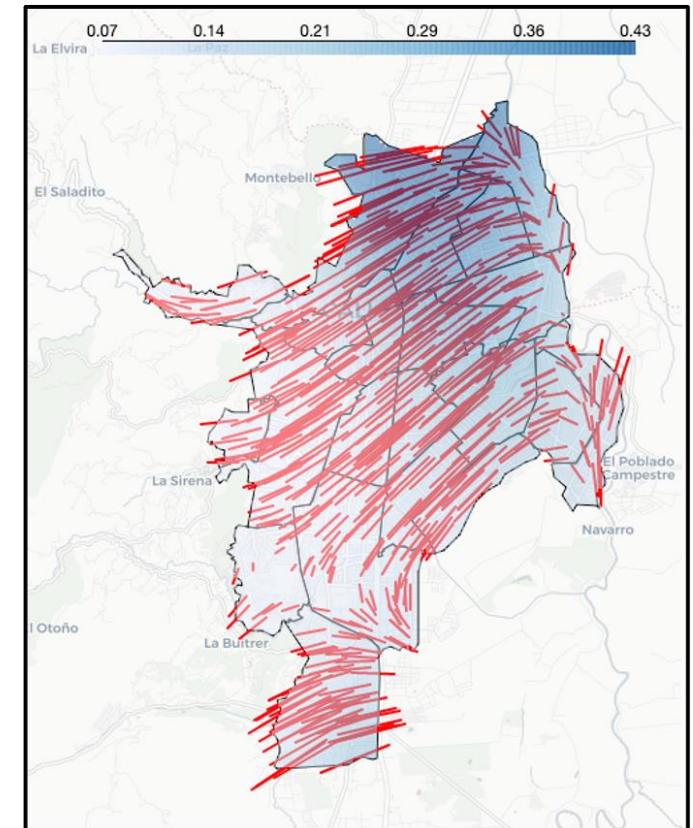
$$v(s, s') = \langle \sum_{r=1}^R w_s^{(r)} \kappa_s^{(r)}(\cdot), \sum_{r=1}^R w_{s'}^{(r)} \kappa_{s'}^{(r)}(\cdot) \rangle$$



$\kappa_S^{(1)}$



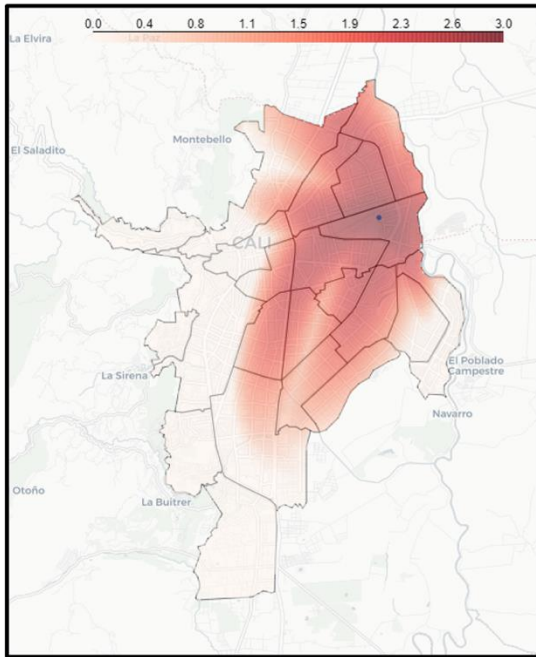
$\kappa_S^{(2)}$



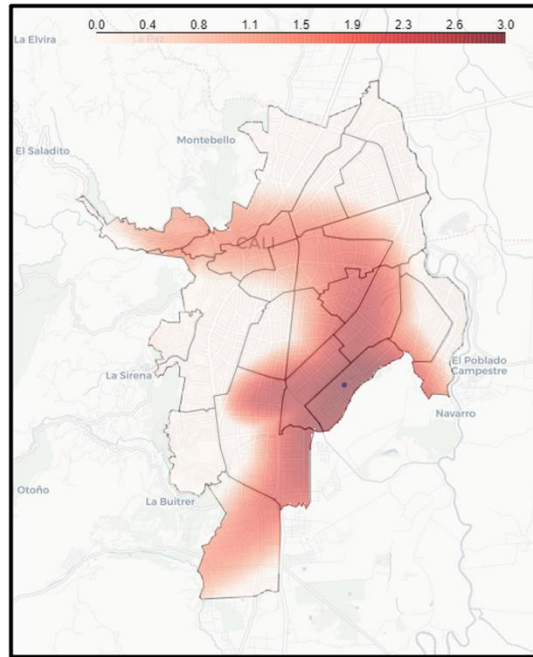
$\kappa_S^{(3)}$

Kernel interpretation – Spatial Kernel

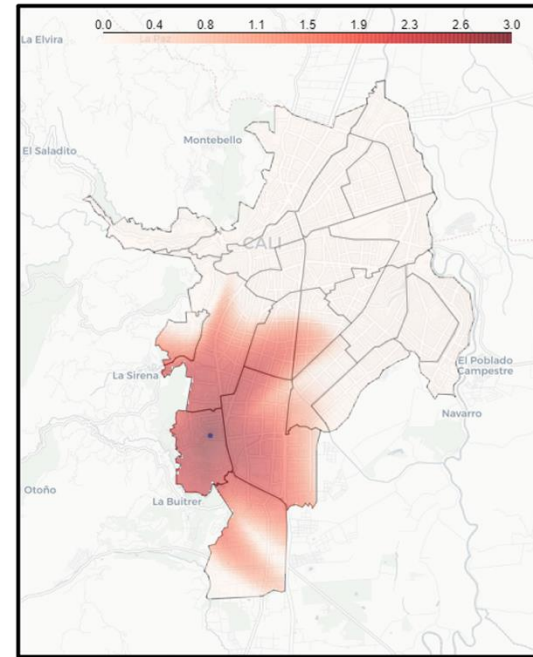
Evaluation of spatial kernel $\nu(s, \cdot)$ over space with one location fixed at s .



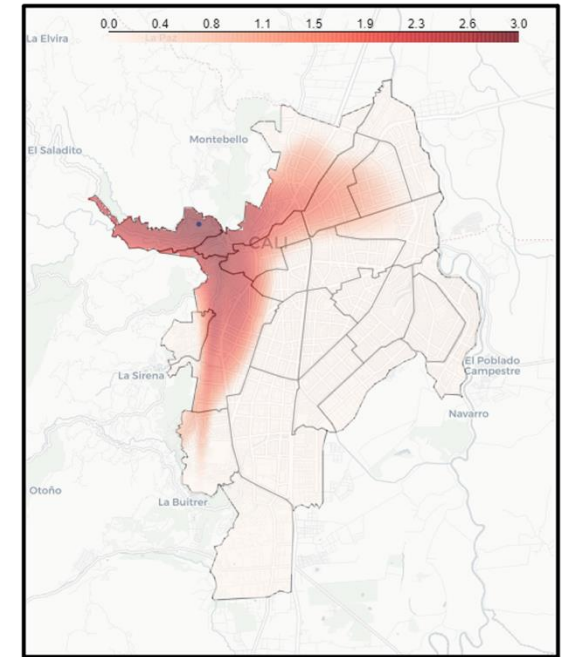
Airport



Center of comuna 15

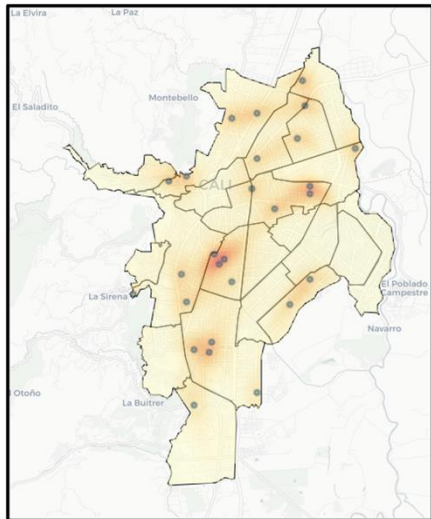


Center of comuna 18

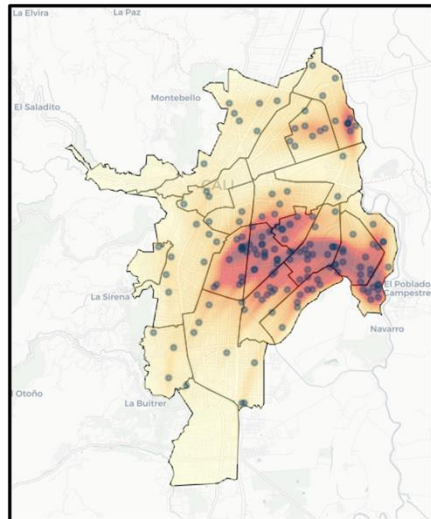


Center of comuna 1

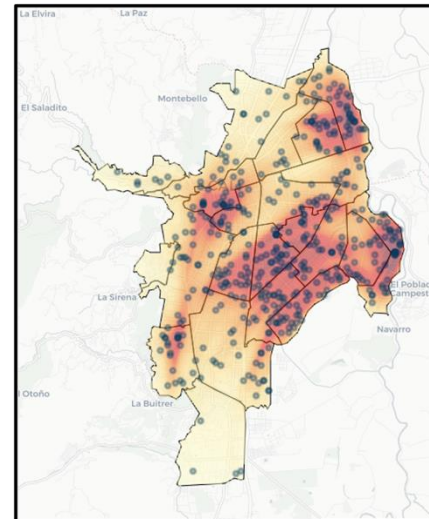
Out-of-sample prediction



03/22/2020



05/17/2020



06/28/2020

Models	MAE $Q_{0.25}^{\text{out}}$	MAE $Q_{0.5}^{\text{out}}$	MAE $Q_{0.75}^{\text{out}}$
Random	10.000	19.000	28.000
SIR	7.101	12.937	18.809
AR(3)	3.332	6.571	12.433
ETAS	4.908 _(0.820)	12.143 _(0.848)	22.238 _(0.852)
NSSTPP-Exo($R=1$)	4.783 _(0.661)	8.944 _(0.748)	15.834 _(0.843)
NSSTPP-Exo($R=2$)	4.710 _(0.643)	9.038 _(0.722)	15.804 _(0.816)
NSSTPP-Exo($R=3$)	4.548 _(0.648)	8.229 _(0.734)	15.750 _(0.809)
NSSTPP($R=3$)	2.843 _(0.585)	6.398 _(0.751)	10.495 _(0.892)

5. Estimating likelihood functions in point process models using neural networks

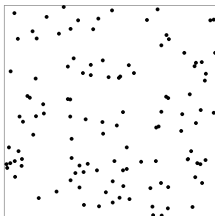
Definition

A **spatial point process** is a stochastic process whose realizations are point patterns in a bounded region $W \subset \mathbb{R}^2$. There are three key formulations:

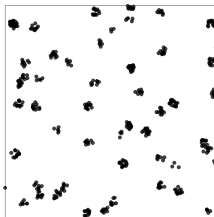
- ① **Density-based:** First, choose $n \sim p_n$, then draw points $x_1, \dots, x_n$ from a joint density $f_n(x_1, \dots, x_n)$. Valid only for finite point patterns.
- ② **Binary grid approximation:** Model the presence/absence in pixels j via binary variables I_j , and treat the pattern as a joint distribution over these.
- ③ **Measure-theoretic:** Treat the process as a random integer-valued measure $N(B)$ over test sets B , satisfying:

$$N(B) \in \mathbb{Z}_{\geq 0}, \quad \forall B \subset \mathbb{R}^2$$

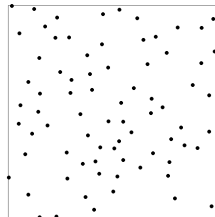
Examples



Homogeneous



Matérn



Strauss

Clustering Processes

1. Log-Gaussian Cox Process (LGCP):

- A doubly stochastic process where the intensity function $\lambda(s)$ follows a log-Gaussian random field.
- Captures overdispersion and spatial heterogeneity.
- Points cluster around high-intensity areas.

2. Thomas Process:

- Cluster process with parent points generated by a Poisson process.
- Offspring points are normally distributed around each parent.
- Results in circular clusters with random centers.

3. Matérn Cluster Process:

- Similar to Thomas, but offspring are uniformly distributed in a disc around each parent.
- Simpler spatial structure with fixed cluster radius.

Inhibitory (Gibbs) Processes

Strauss Process:

- A Gibbs process with pairwise interaction between points.
- Characterized by a hard-core or soft inhibition radius R and interaction parameter γ .
- Probability density (w.r.t. Poisson) given by:

$$f(x_1, \dots, x_n) \propto \beta^n \gamma^{s(x_1, \dots, x_n)}$$

where $s(\cdot)$ is the number of close pairs with distance less than R .

- $\gamma < 1$ induces inhibition (repulsion); $\gamma = 1$ reduces to Poisson.

Intractable Likelihoods

Likelihood in Spatial Point Processes

In general, a spatial point process over a bounded window $W \subset \mathbb{R}^2$ has a likelihood of the form:

$$L(\theta \mid \mathbf{x}) = f_{\theta}(x_1, \dots, x_n) = \frac{1}{Z(\theta)} f(x_1, \dots, x_n)$$

- f : potential function (may encode interaction, aggregation, or both)
- $Z(\theta)$: normalizing constant ensuring the total probability integrates to 1

Issue: In most practical models, $Z(\theta)$ is unknown and intractable.

Why is the Likelihood Intractable?

The normalizing constant is defined as:

$$Z(\theta) = \sum_{n=0}^{\infty} \frac{1}{n!} \int_{W^n} (f(x_1, \dots, x_n)) dx_1 \dots dx_n$$

Difficulties:

- The integral is over an *infinite-dimensional, unordered configuration space*.
- It involves all possible point counts and locations.
- The function f may include complex spatial interactions or latent components (as in LGCP).

Conclusion: $Z(\theta)$ cannot be computed analytically nor efficiently approximated in general.

Models with Intractable Likelihoods

This issue arises in a wide variety of spatial point process models:

1. Cluster processes: Thomas, Matérn, Log-Gaussian Cox Process (LGCP)

$$f(x) = \int f(x \mid \Lambda) p(\Lambda \mid \theta) d\Lambda$$

- Likelihood is a marginal over a latent random intensity field $\Lambda(u)$.
- No closed-form marginal density: $Z(\theta)$ implicit in the integral.

2. Gibbs models: Strauss, Geyer, Area-interaction

$$f(x) \propto \beta^n \gamma^{s(x)} \quad \text{with unknown } Z(\beta, \gamma)$$

Implications for Statistical Inference

Consequences of intractable likelihoods:

- Maximum likelihood estimation becomes unavailable.
- Bayesian inference (posterior computation) is intractable.
- Model comparison via AIC/BIC or marginal likelihood is compromised.

Practically: We must rely on alternative inference strategies:

- Pseudo-likelihood and composite likelihood.
- Simulation-based methods (Monte Carlo, MCMC, ABC).
- Classification-based surrogates or contrastive approaches.

Composite Likelihood for Point Patterns

Definition: A composite likelihood is formed by multiplying likelihoods of lower-dimensional marginal events.

$$CL(\theta \mid \mathbf{x}) = \prod_{(x_i, x_j) \in \mathbf{x}} f_{2,\theta}(x_i, x_j)$$

- Typically uses all pairs (or k -tuples) of points.
- $f_{2,\theta}(x_i, x_j)$: bivariate marginal density.
- Trades off model fidelity for computational tractability.

Advantage: Avoids evaluating full joint density or intractable normalizing constants.

Pseudo-likelihood for Point Processes

Definition (Besag, 1974): The pseudo-likelihood approximates the true likelihood by assuming local independence.

$$PL(\theta \mid \mathbf{x}) = \prod_{x_i \in \mathbf{x}} \lambda_{\theta}(x_i \mid \mathbf{x} \setminus x_i) \cdot \exp \left(- \int_W \lambda_{\theta}(u \mid \mathbf{x}) du \right)$$

- $\lambda_{\theta}(u \mid \mathbf{x})$: conditional intensity at location u given the rest of the pattern.
- Easier to compute than full likelihood since no $Z(\theta)$ is needed.

Usage: Widely used in spatial statistics for Gibbs models.

Likelihood-Free Inference via CNN

Data Generation and Classifier Training

Step 1: Build the training dataset

- Sample parameters $\theta_1, \dots, \theta_m$ (e.g., via Latin Hypercube Sampling).
- Simulate n realizations $y_{i,1}, \dots, y_{i,n} \sim p(y \mid \theta_i)$.
- Create null class $\mathcal{C}_2$ by permuting:

$$\mathcal{C}_2 = \{(y_{i,j}, \theta_{\pi_j(i)})\}$$

Step 2: Train binary classifier $h(y, \theta)$ to distinguish:

$$\begin{cases} (y, \theta) \sim p(y \mid \theta)p(\theta) & \text{(Class 1)} \\ (y, \theta) \sim p(y)p(\theta) & \text{(Class 2)} \end{cases}$$

Goal: Learn $h(y, \theta) = P(C = 1 \mid (y, \theta))$ using the simulated data.

Likelihood Approximation from the Classifier

Connection to Likelihood:

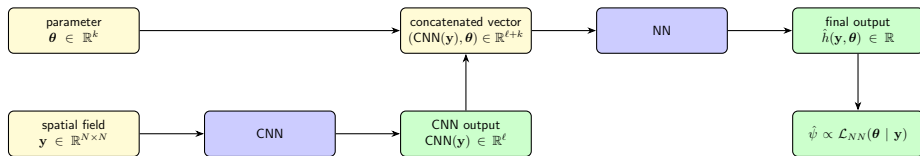
$$h(y, \theta) = \frac{\frac{p(y|\theta)}{p(y)p(\theta)}}{\frac{p(y|\theta)}{p(y)p(\theta)} + 1} = \frac{L(\theta | y)/p(y)}{L(\theta | y)/p(y) + 1} \Rightarrow L(\theta | y) \propto \frac{h(y, \theta)}{1 - h(y, \theta)}$$

For multiple i.i.d. observations:

$$L(\theta | y_1, \dots, y_n) \propto \prod_{i=1}^n \frac{h(y_i, \theta)}{1 - h(y_i, \theta)}$$

Conclusion: The classifier acts as a likelihood proxy, bypassing the intractable normalizing constant $Z(\theta)$.

CNN-Based Likelihood Proxy



Architecture:

- Convert each realization y to a $N \times N$ grid of point counts.
- Pass through CNN layers to extract spatial structure.
- Concatenate CNN output with the parameter vector θ .
- Final layers: fully connected network producing $h(y, \theta) \in [0, 1]$.

Grid Resolution Trade-off:

- Higher resolution captures more spatial information.
- However, it increases sparsity, memory, and training time.

Convolutional Neural Network Architecture

Layer Type	Output Shape	Filters	Kernel Size	Activation	Weights
2D convolution	(48, 48, 128)	128	(3, 3)	ReLU	1,280
MaxPooling2D	(24, 24, 128)	-	(2, 2)	-	0
2D convolution	(22, 22, 128)	128	(3, 3)	ReLU	147,584
MaxPooling2D	(11, 11, 128)	-	(2, 2)	-	0
2D convolution	(9, 9, 128)	128	(3, 3)	ReLU	147,584
MaxPooling2D	(5, 5, 128)	-	(2, 2)	-	0
2D convolution	(3, 3, 16)	16	(3, 3)	ReLU	18,448
MaxPooling2D	(2, 2, 16)	-	(2, 2)	-	0
Flatten	(64,)	-	-	-	0
Concatenate	(66,)	-	-	-	0
Dense	(64,)	-	-	ReLU	4,288
Dense	(16,)	-	-	ReLU	1,040
Dense	(8,)	-	-	ReLU	136
Dense	(2,)	-	-	Softmax	18

Parameter Inference via GNN

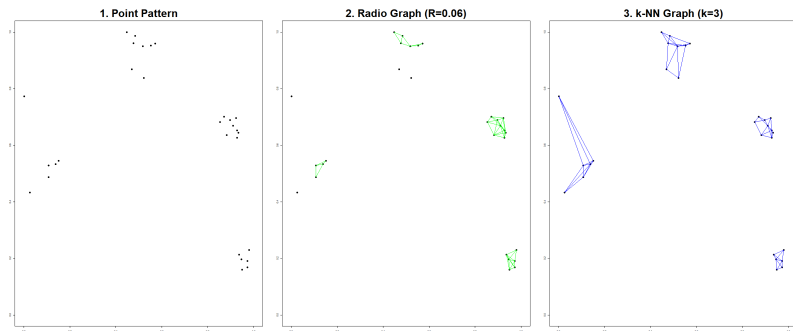
Beyond CNN: Toward GNNs

- CNNs require discretization
- Graph Neural Networks (GNNs) can operate on raw coordinates
- GNNs preserve geometric properties and could generalize better

Data Generation and GNN Training

Step 1: Simulate training dataset

- Sample parameters $\theta_1, \dots, \theta_m$ from prior $p(\theta)$.
- For each θ_i , simulate a realization $y_i \sim p(y \mid \theta_i)$.
- Convert each y_i into a graph G_i (e.g., via k -NN or radius graph).
- Store training pairs (G_i, θ_i) .



Data Generation and GNN Training

Step 2: Train a regression model $f(G)$

- Use a GNN to map graphs G_i to parameter estimates $\hat{\theta}_i$.
- Minimize the mean squared error (MSE):

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m \|f(G_i) - \theta_i\|^2$$

Goal: Learn $f(G)$ to predict θ from new point patterns y (converted to G).

Graph Neural Network Architecture (GNN)

Layer Type	Input Features	Output Features	Operation
Graph Convolution Layer	d	32	Aggregation + Linear + ReLU
Graph Convolution Layer	32	32	Aggregation + Linear + ReLU
Graph Convolution Layer	32	32	Aggregation + Linear + ReLU
Global Mean Pooling	variable #nodes	32	
Fully Connected (Linear)	32	2	Linear projection

Example 1: Small Graph (10 nodes)

- Each node has d features (e.g., location, local density).
- Nodes exchange information with neighbors (via edges).
- After 3 GCN layers: each node has 32-dimensional embedding.
- Global mean pooling averages embeddings $\rightarrow$ 1 vector of size 32.
- Final output: 2 predicted values.

Example 2: Large Graph (100 nodes)

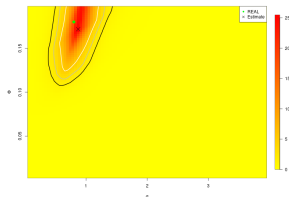
- Same feature format per node.
- Much more information passed between more neighbors.
- After 3 GCN layers: each node has 32-dimensional embedding.
- Global mean pooling averages across 100 nodes $\rightarrow$ 1 vector of size 32.
- Final output: 2 predicted values.

Each graph may have a different number of nodes, but the global pooling ensures a fixed-size output per graph (independent of its size).

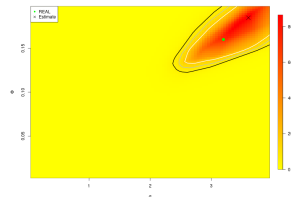
Results: CNN vs Composite Likelihood

Evaluation with CNN

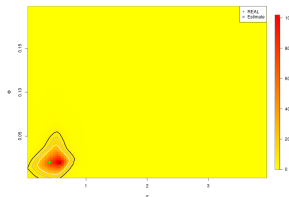
LGCP Model



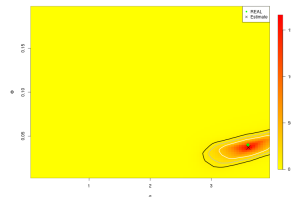
(a) $\sigma = 0.8$, $\phi = 0.18$



(b) $\sigma = 3.2$, $\phi = 0.16$



(c) $\sigma = 0.4$, $\phi = 0.02$

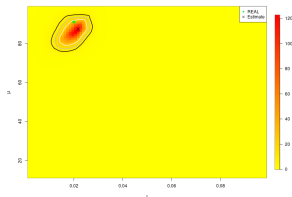


(d) $\sigma = 3.6$, $\phi = 0.04$

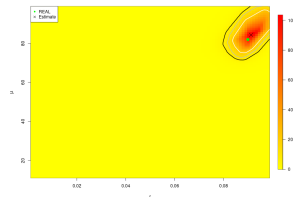
Figure: Neural log-likelihood estimates in four different simulations of the LGCP model.

Evaluation with CNN

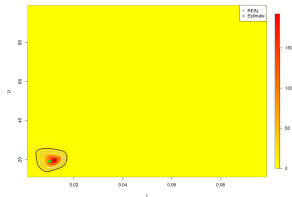
Matérn Model



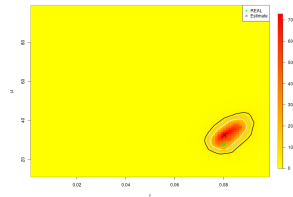
(a) $r = 0.02$, $\mu = 91$



(b) $r = 0.09$, $\mu = 82$



(c) $r = 0.01$, $\mu = 19$

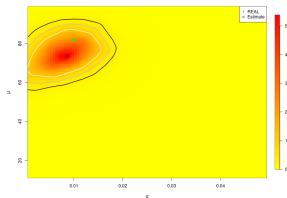


(d) $r = 0.08$, $\mu = 28$

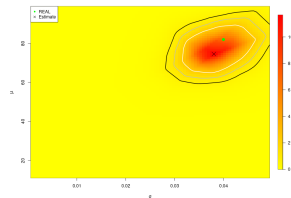
Figure: Neural log-likelihood estimates in four different simulations of the Matérn model.

Evaluation with CNN

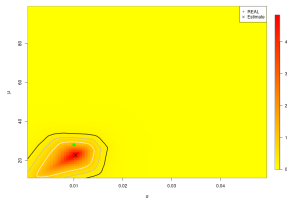
Thomas Model



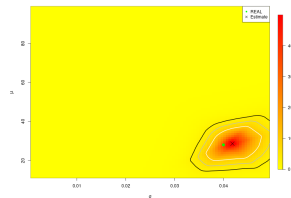
(a) $\sigma = 0.01, \mu = 82$



(b) $\sigma = 0.04, \mu = 16$



(c) $\sigma = 0.01, \mu = 28$



(d) $\sigma = 0.04, \mu = 28$

Figure: Neural log-likelihood estimates in four different simulations of the Thomas model.

Computation Time: CNN vs Classical Method

Comparison of estimation speed and accuracy

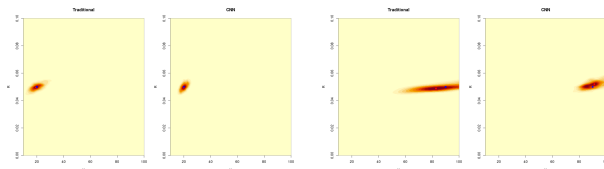


Figure: Comparison between the densities of the estimated parameters using the traditional method versus the CNN.

Model	Mean N ^o Points	CNN (sec)	Composite Likelihood (sec)	Speedup
Matérn (1)	2000	1.6642	0.984362	x0.5915
Matérn (2)	9000	1.8092	17.71762	x9.793
LGCP	3000	1.6941	2.210772	x1.305

Table: Average estimation time per pattern (CPU).

Coverages

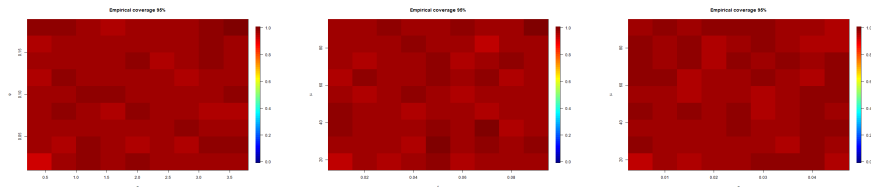


Figure: Empirical coverage at 95% for the LGCP (left), Matérn (center), and Thomas (right) models across 81 parameter settings and 200 simulations each.

Evaluation with GNN

Matérn Model

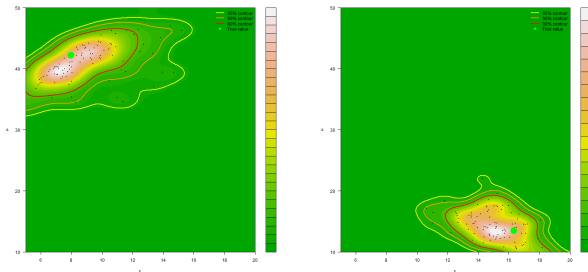


Figure: Density obtained from estimates over 100 samples with the same parameters using the Matérn model trained with a GNN.

Evaluation with GNN

Strauss Model

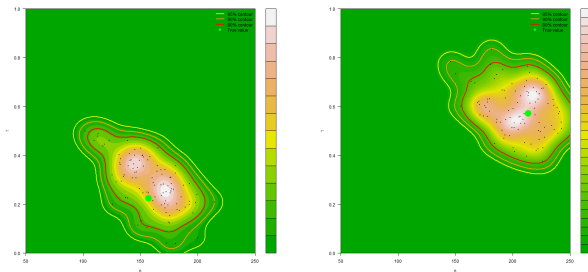


Figure: Density obtained from estimates over 100 samples with the same parameters using the Strauss model trained with a GNN.

Computation Time: GNN vs Classical Method

Comparison of estimation speed and accuracy

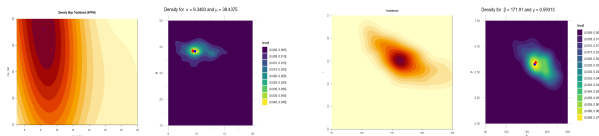


Figure: Comparison between the densities of the estimated parameters using the traditional method versus the GNN. On the left for the Matérn model (clustering) and on the right for the Strauss model (inhibition).

Model	Mean N ^o Points	GNN (sec)	Traditional (Composite or Pseudo)	Speedup
Matérn	1000	0.003153	0.9174	x290.961
Strauss	100	0.001178	0.009	x7.64

Table: Average estimation time per pattern (CPU).

Coverages

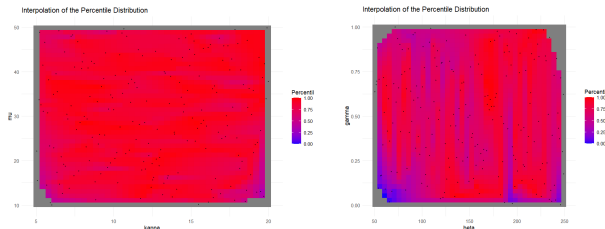


Figure: Percentiles of the True Value within the Posterior Density Estimated from 200 GNN-Based Parametric Inferences: Matérn Model (Left, Clustering) and Strauss Model (Right, Inhibition)

Thank you for your attention!